

Intelligence Artificielle Guidée par les Lois Scientifiques

Quand les modèles pilotés par les données apprennent à respecter les lois de la science

Mohamed Youssfi, Enseignant Chercheur, ENSET Mohammedia, Directeur Laboratoire Informatique, Intelligence Artificielle et Cyber Sécurité

<https://www.youtube.com/watch?v=jq04Mkid5s0>

<https://github.com/mohamedYoussfi/pinns-demo>



International conference on statistics and data sciences

ICAMDS & SFC

2nd Edition 31st Edition

Mathematics • Artificial Intelligence • Computer Science

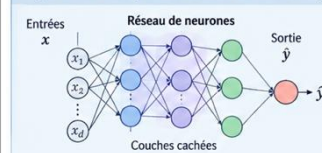
Casablanca, 14–18 September 2026

INTERNATIONAL CONFERENCE

Applied Mathematics & Data Science 2nd Edition

Société Francophone De Classification 31st Edition

1. Modèles pilotés par les données (Data-Driven, ex. réseaux de neurones)

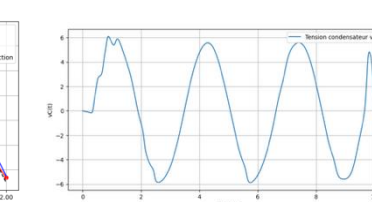
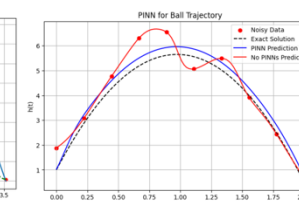
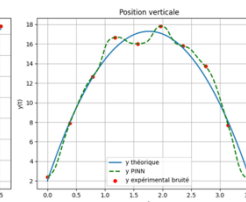
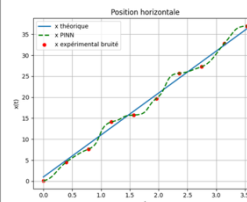


Limites

- Données parfois insuffisantes
- Données bruitées
- Risque d'overfitting
- Coût élevé de collecte des données
- Généralisation limitée
- Résultats parfois physiquement irréalistes (ex. : générer une voiture qui vole ...)



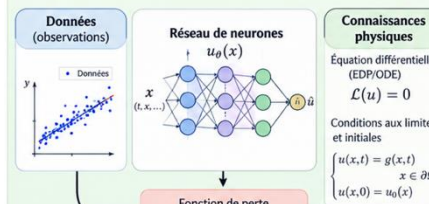
Un modèle purement piloté par les données peut produire des résultats irréalistes.



2. PINN : modèle hybride

Piloté par les données et informé par les lois physiques

Idee : entraîner un réseau de neurones en utilisant à la fois les données et les équations de la physique (EDP/ODE, conditions aux limites, etc.).



Fonction de perte $\mathcal{L} = \mathcal{L}_{data} + \lambda \mathcal{L}_{phys}$

- Le modèle physique améliore l'IA**
 - ✓ Filtre l'effet du bruit
 - ✓ Réduit le surapprentissage
 - ✓ Impose une cohérence scientifique
 - ✓ Permet d'apprendre avec moins de données
- L'IA améliore le modèle physique**
 - ✓ Apprend les phénomènes négligés par les hypothèses
 - ✓ Compense les approximations
 - ✓ Identifie des paramètres inconnus
 - ✓ Fournit une approximation rapide de solutions complexes

Modèle hybride plus fiable et plus performant

- ✓ Prédictions cohérentes avec la physique
- ✓ Meilleure généralisation
- ✓ Corrige les limites des approches classiques
- ✓ Applicable à la physique, la finance, la santé, ...

3. Modèles purement physiques (modèles mathématiques)

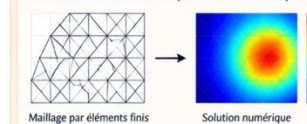
Exemple : équation de la chaleur (EDP)

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u \quad \text{dans } \Omega$$

$$u(x, t) = g(x, t) \quad \text{sur } \partial\Omega$$

$$u(x, 0) = u_0(x)$$

Méthodes de résolution (ex. éléments finis)



Limites

- ✗ Hypothèses simplificatrices
- ✗ Phénomènes parfois négligés
- ✗ Modèles difficiles à résoudre analytiquement
- ✗ Méthodes numériques (éléments finis, etc.) très coûteuses en calcul
- ✗ Modèles parfois très approximatifs
- ✗ Élaboration complexe et chronophage
- ✗ Difficile à adapter à des systèmes multi-physiques ou très complexes



Intelligence Artificielle

l'IA qui raisonne à partir de **règles explicites**

l'IA qui apprend à partir de **données, d'expérience ou de contraintes physiques**.

IA Symbolique

Approche historique et toujours utilisée

Une approche de l'IA qui

- représente les connaissances sous forme de symboles, de règles explicites et de logique formelle.
- Adaptée aux systèmes déterministes
- Utilisée dans systèmes experts et les moteurs de règles
- IA Explicable, contrôlable et prévisible

IA Basée sur l'apprentissage (Machine Learning)

Approche très dominante

IA Prédictive et décisionnelle

Apprendre pour prédire et prendre des décisions à partir des **données structurées ou non structurées**, de l'**expérience** ou de la **physique**

Modèles Pilotés par les données

Apprentissage Supervisé

Apprentissage Non Supervisé

Besoin de données en quantité et en qualité

Algorithmes :

- SVM, DT, RF, KNN, XGBoost, etc.
- **Neural Network** (ANN, CNN, RNN, LSTM)

Modèles Pilotés par l'expérience

Apprentissage Par renforcement



Besoin une abstraction de l'environnement

- Q-learning, Deep Q-learning
- PPO
- **Neural Network**

Modèles Guidés par les lois de la physique

PINN

Besoin des lois de la, physique et des données

Algorithmes :

- **Neural Network**

IA Générative

Apprend les structures des données, souvent non structurées, pour générer de nouveaux contenus réalistes : texte, image, son, vidéo, code.

Apprentissage Auto Supervisé, LLMs

Algorithmes :

- **Neural Network** (Transformers, GAN, ...)

IA Agentique

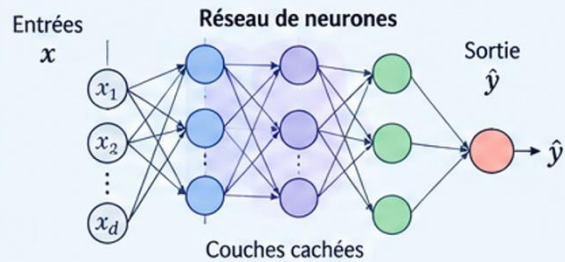
Architecture dans laquelle un agent **perçoit son environnement**, **raisonne**, **planifie**, **utilise des outils** et **agit** pour atteindre un objectif.

Technologies :

- LLM (**Neural Network**)
- Tools, MCP
- Planification, RAG,
- Apprentissage par renforcement

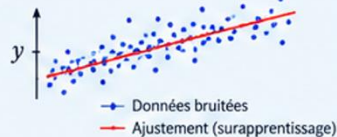
Problématique : Vers des modèles hybrides informés par la physique

1. Modèles pilotés par les données (Data-Driven, ex. réseaux de neurones)

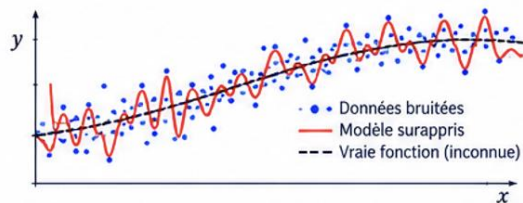


Limites

- Données parfois insuffisantes
- Données bruitées



- Risque d'overfitting



- Coût élevé de collecte des données
- Généralisation limitée
- Résultats parfois physiquement irréalistes (ex. : générer une voiture qui vole ...)

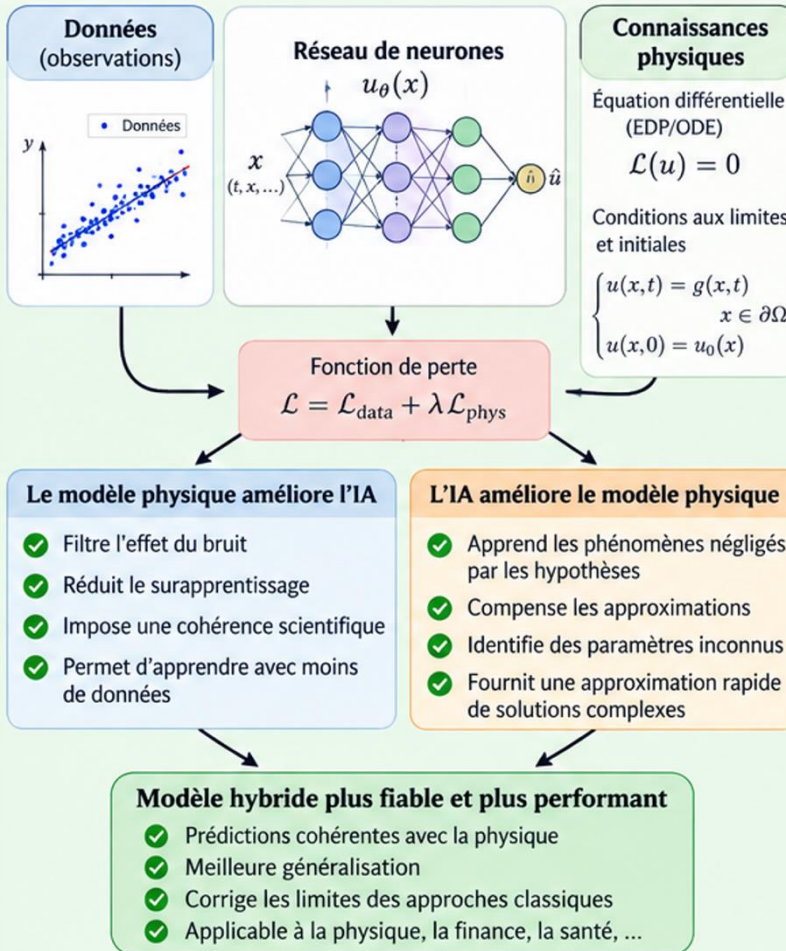


Un modèle purement piloté par les données peut produire des résultats irréalistes.

2. PINN : modèle hybride

Piloté par les données et informé par les lois physiques

Idee : entraîner un réseau de neurones en utilisant à la fois les données et les équations de la physique (EDP/ODE, conditions aux limites, etc.).



3. Modèles purement physiques (modèles mathématiques)

Exemple : équation de la chaleur (EDP)

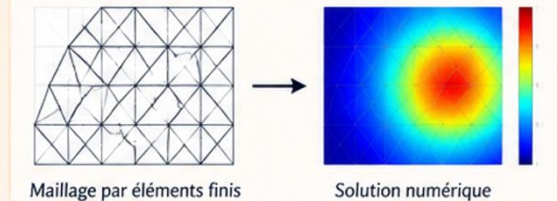
$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u \quad \text{dans } \Omega$$
$$u(x, t) = g(x, t) \quad \text{sur } \partial\Omega$$
$$u(x, 0) = u_0(x)$$

$u(x, t)$

x

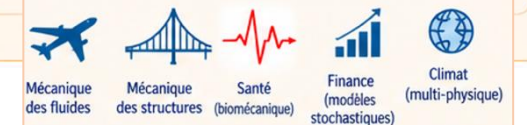
t

Méthodes de résolution (ex. éléments finis)



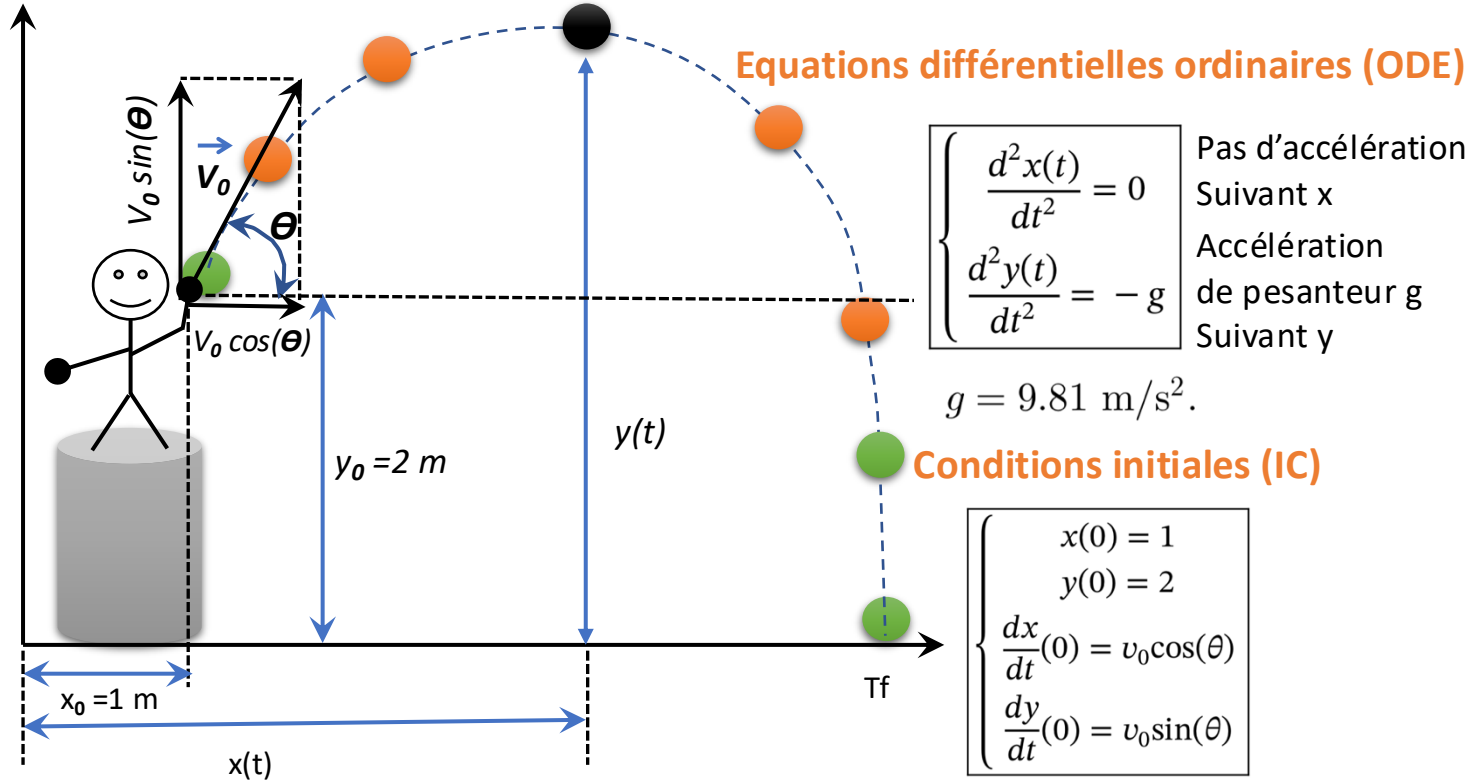
Limites

- Hypothèses simplificatrices
- Phénomènes parfois négligés
- Modèles difficiles à résoudre analytiquement
- Méthodes numériques (éléments finis, etc.) très coûteuses en calcul
- Modèles parfois très approximatifs
- Élaboration complexe et chronophage
- Difficile à adapter à des systèmes multi-physiques ou très complexes



PINN = Apprendre à partir des données sans oublier les lois qui gouvernent le système

Physics based Model



Hypothèses simplificatrices :

- Projectile assimilé à un point matériel
- Absence de résistance de l'air
- Absence de vent
- Champ de pesanteur uniforme et constant : $g=9,81 \text{ m/s}^2$.
- Terre supposée localement plane
- Rotation de la Terre négligée
- Mouvement dans un plan 2D
- Masse du projectile constante pendant tout le mouvement.
- Pas de rotation du projectile : effets de spin/Magnus négligés.
- Aucune propulsion après le lancement : après $t=0$, seule la gravité agit.
- Conditions initiales parfaitement connues

$$\sum \vec{F} = m\vec{a} \quad \vec{P} = m\vec{g}$$

$$\sum F_x = 0 \quad \sum F_y = -mg$$

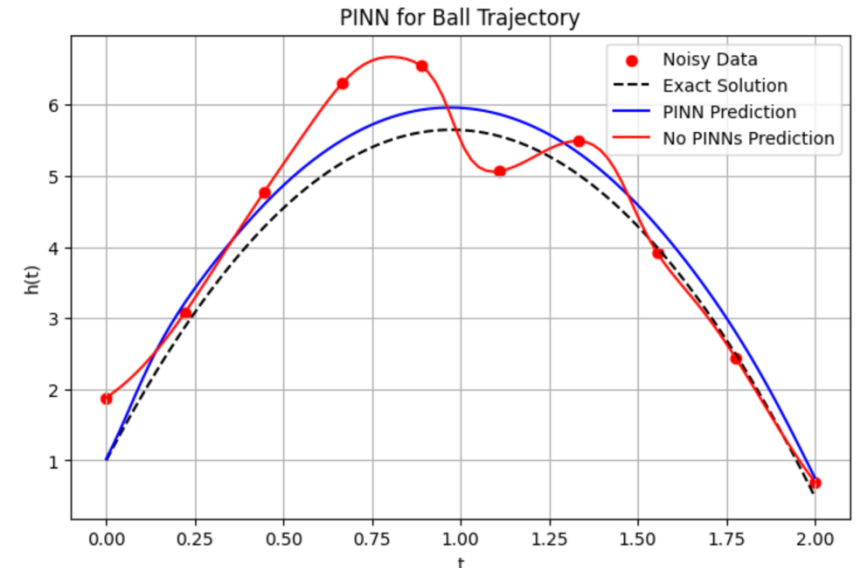
$$m \frac{d^2x}{dt^2} = 0 \quad m \frac{d^2y}{dt^2} = -mg$$

$$\boxed{\frac{d^2x}{dt^2} = 0} \quad \boxed{\frac{d^2y}{dt^2} = -g}$$

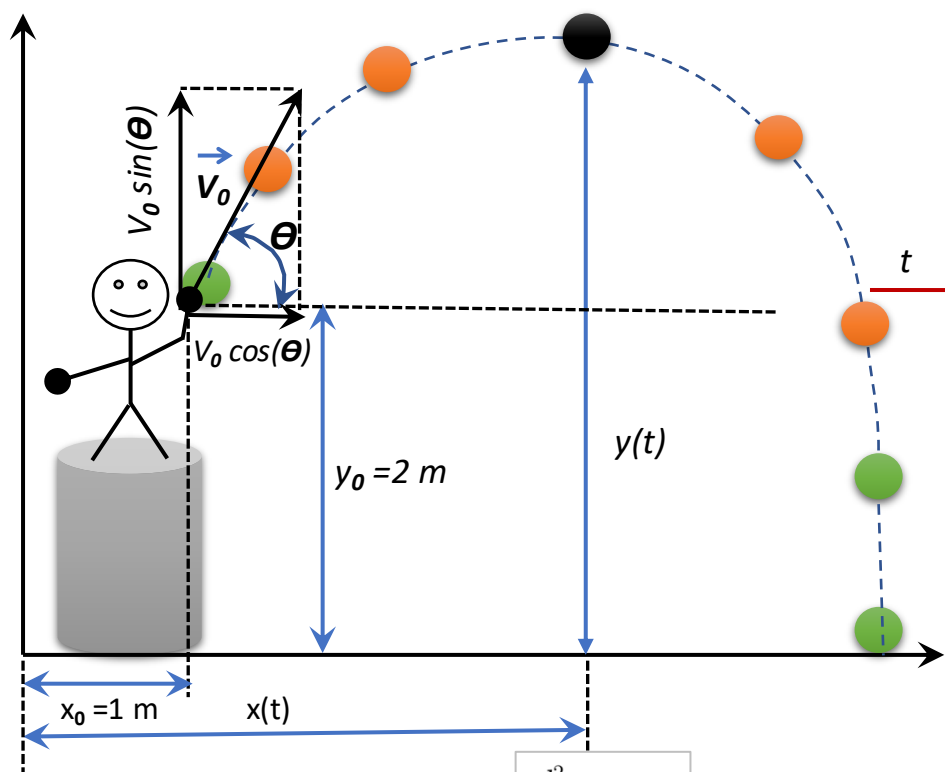
Solution analytique :

$$x(t) = x_0 + v_0 \cos(\theta) t$$

$$y(t) = y_0 + v_0 \sin(\theta) t - \frac{1}{2} gt^2$$



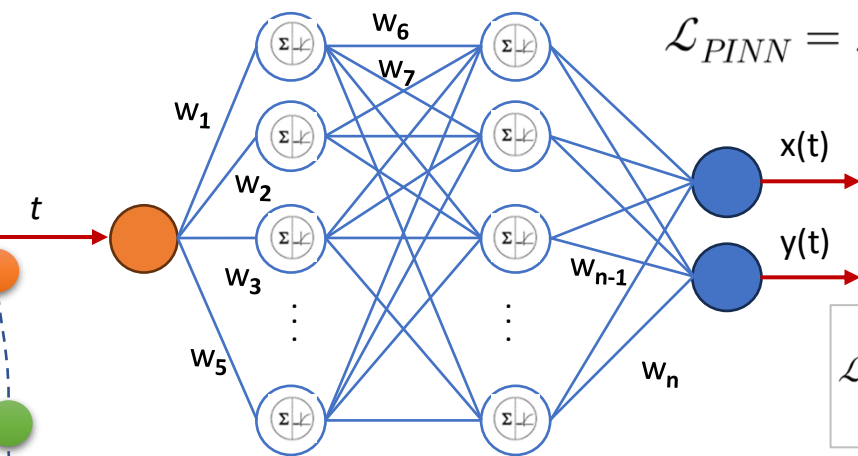
Physics Informed Neural Network



	time	X	y
0	0.000000	0.471534	2.219682
1	0.588532	9.586019	15.523645
2	1.177065	17.993155	25.836216
3	1.765597	28.226082	33.179218
4	2.354130	35.791260	35.383904
5	2.942662	44.638611	36.379066
6	3.531194	54.629513	32.692162
7	4.119727	62.136314	26.421669
8	4.708259	71.856407	15.405033
9	5.296792	80.820435	2.086739

$$\frac{d^2 x}{dt^2} = 0$$
$$\frac{d^2 y}{dt^2} = -g$$

$$x(0) = x_0, \quad y(0) = y_0$$
$$\frac{dx}{dt}(0) = v_0 \cos \theta$$
$$\frac{dy}{dt}(0) = v_0 \sin \theta$$

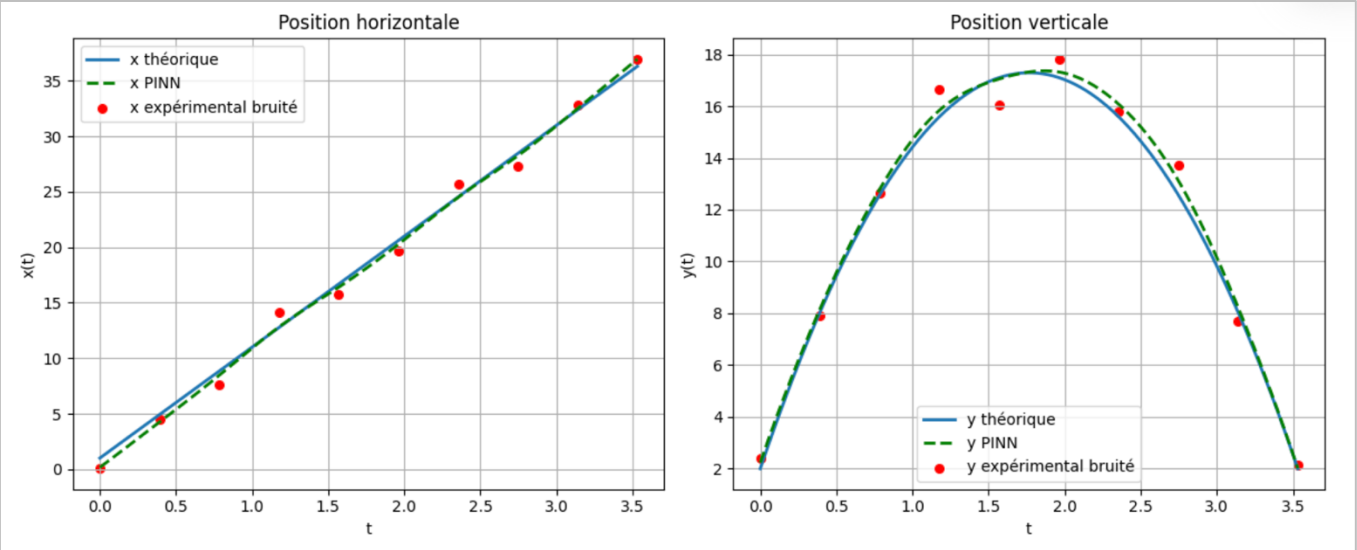


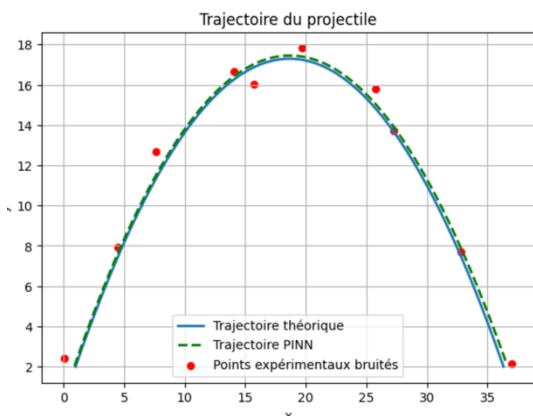
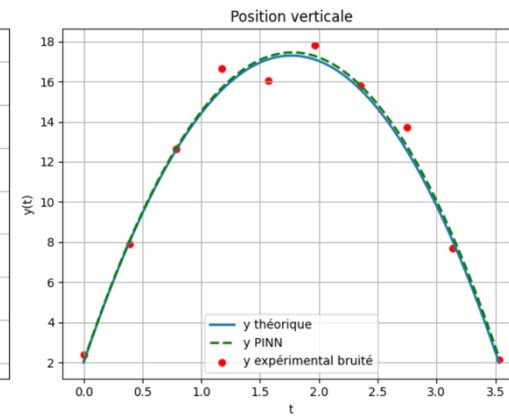
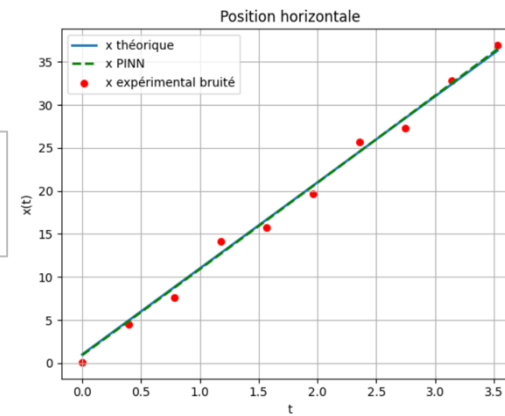
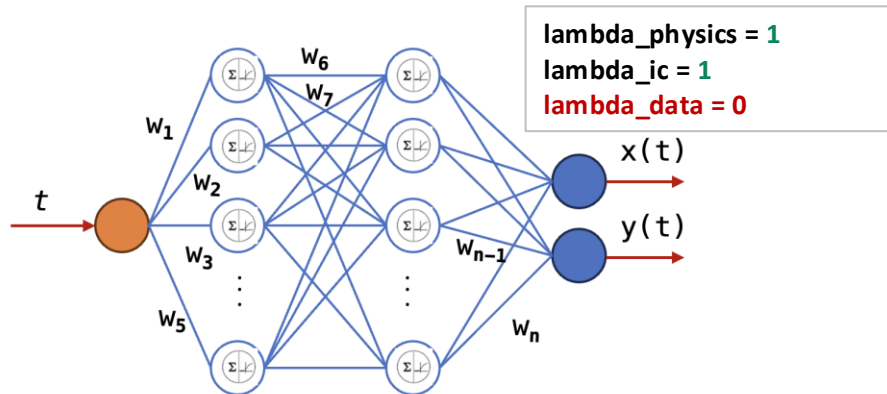
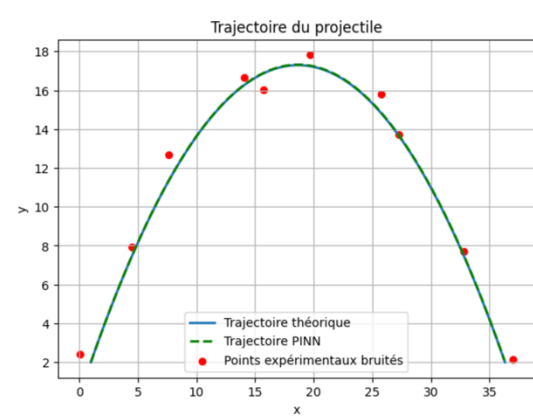
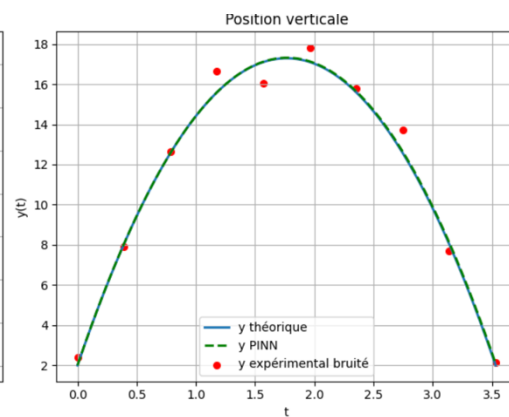
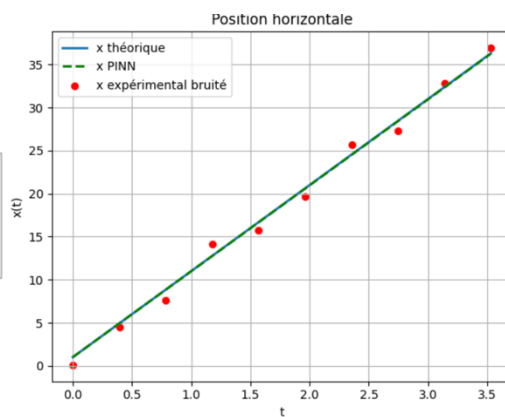
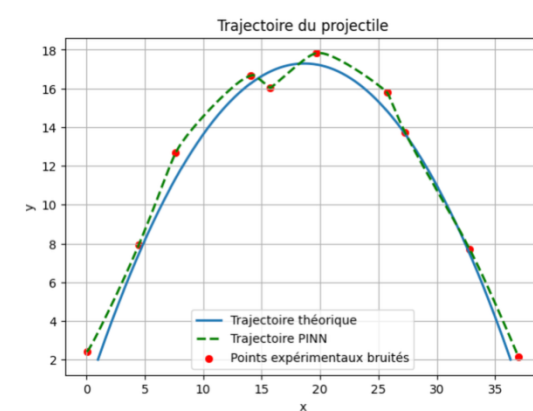
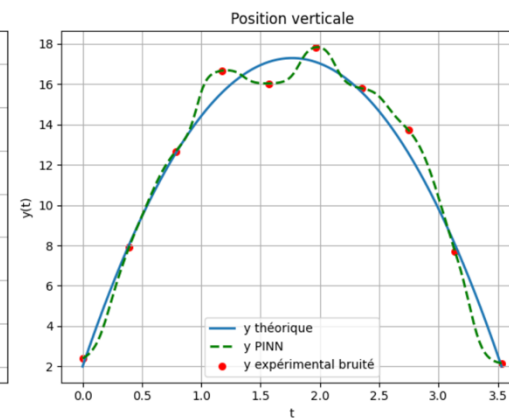
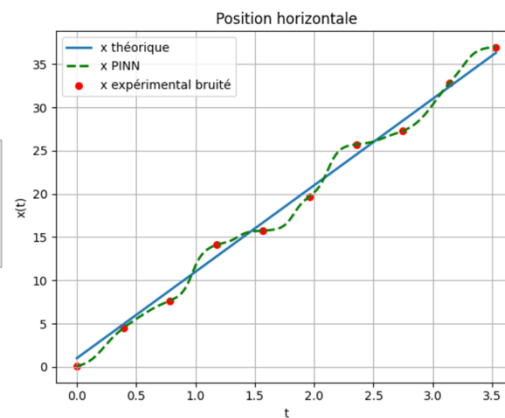
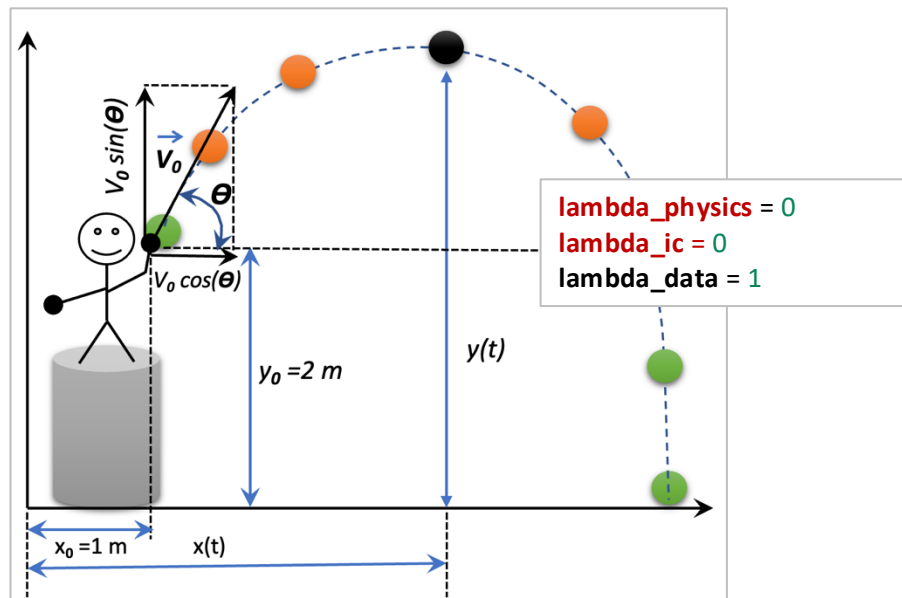
$$\mathcal{L}_{PINN} = \lambda_{data} \mathcal{L}_{data} + \lambda_{physics} \mathcal{L}_{physics} + \lambda_{IC} \mathcal{L}_{IC}$$

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N [(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2]$$

$$\mathcal{L}_{physics} = \frac{1}{N_f} \sum_{i=1}^{N_f} \left[\left(\frac{d^2 \hat{x}(t_i)}{dt^2} \right)^2 + \left(\frac{d^2 \hat{y}(t_i)}{dt^2} + g \right)^2 \right]$$

$$\mathcal{L}_{IC} = (\hat{x}(0) - x_0)^2 + (\hat{y}(0) - y_0)^2 + \left(\frac{d\hat{x}}{dt}(0) - v_0 \cos \theta \right)^2 + \left(\frac{d\hat{y}}{dt}(0) - v_0 \sin \theta \right)^2$$

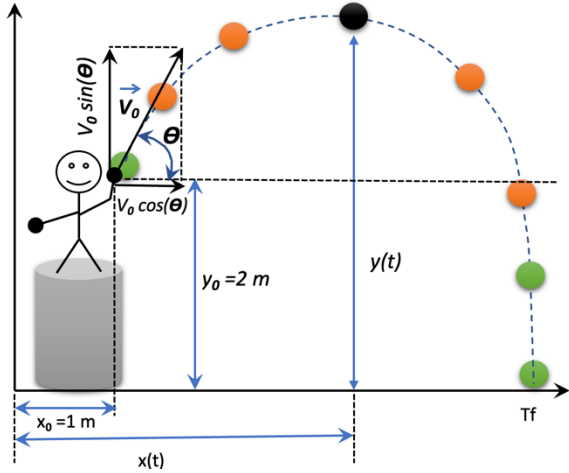




$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$\lambda_{\text{physics}} = 1$
 $\lambda_{\text{ic}} = 1$
 $\lambda_{\text{data}} = 1$

Physics based Model



$$\begin{cases} \frac{d^2x(t)}{dt^2} = 0 \\ \frac{d^2y(t)}{dt^2} = -g \end{cases} \quad \begin{cases} x(0) = 1 \\ y(0) = 2 \\ \frac{dx}{dt}(0) = v_0 \cos(\theta) \\ \frac{dy}{dt}(0) = v_0 \sin(\theta) \end{cases}$$

$$g = 9.81 \text{ m/s}^2.$$

$$x(t) = x_0 + v_0 \cos(\theta) t$$

$$y(t) = y_0 + v_0 \sin(\theta) t - \frac{1}{2} g t^2$$



```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import math
import pandas as pd
torch.manual_seed(42)
# Paramètres physiques
g = 9.81
# Conditions initiales
x0 = 1.0
y0 = 2.0
```

```
teta = math.pi/3
v0 = 20 # m/s
vx0 = v0*math.cos(teta)
vy0 = v0*math.sin(teta)
```

```
def physics_model(t):
    # Solution théorique exacte
    x_exact = x0 + vx0 * t
    y_exact = y0 + vy0 * t - 0.5 * g * t ** 2
    return x_exact, y_exact
```

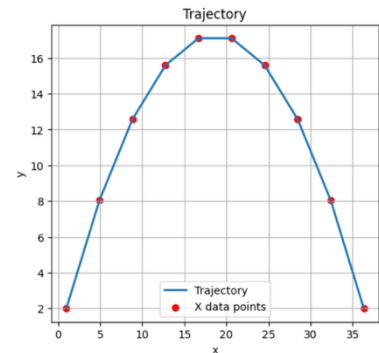
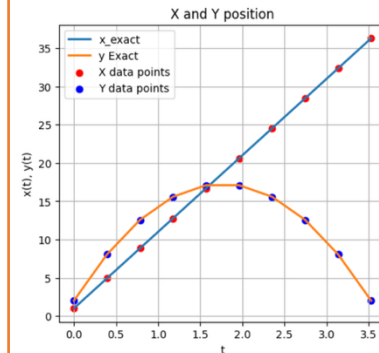
```
# Temps de vol théorique pour y = 2
T_final = 2 * vy0 / g
print("Temps de vol théorique :", T_final)
N_data = 10
t_test = torch.linspace(0, T_final, N_data).view(-1, 1)
```

```
x, y = physics_model(t_test)
```

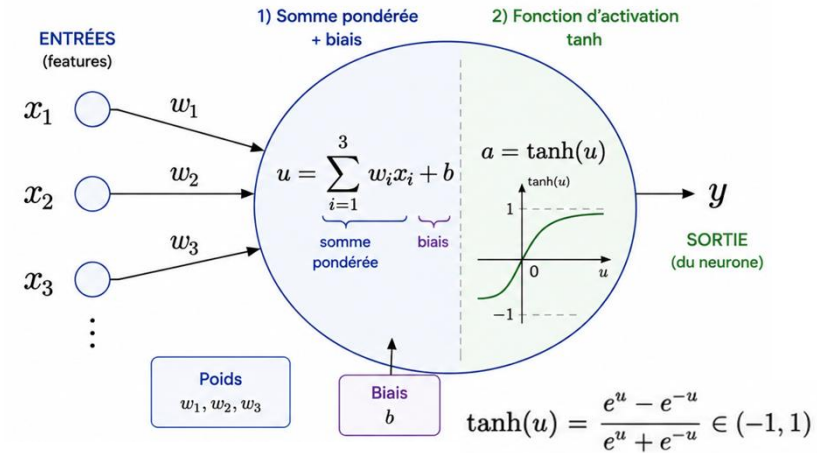
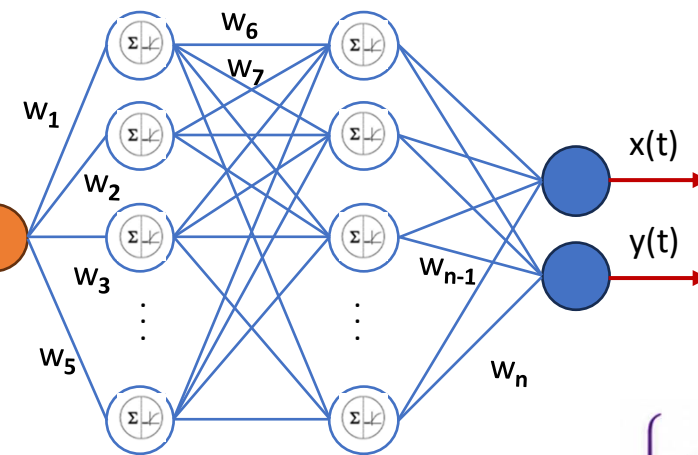
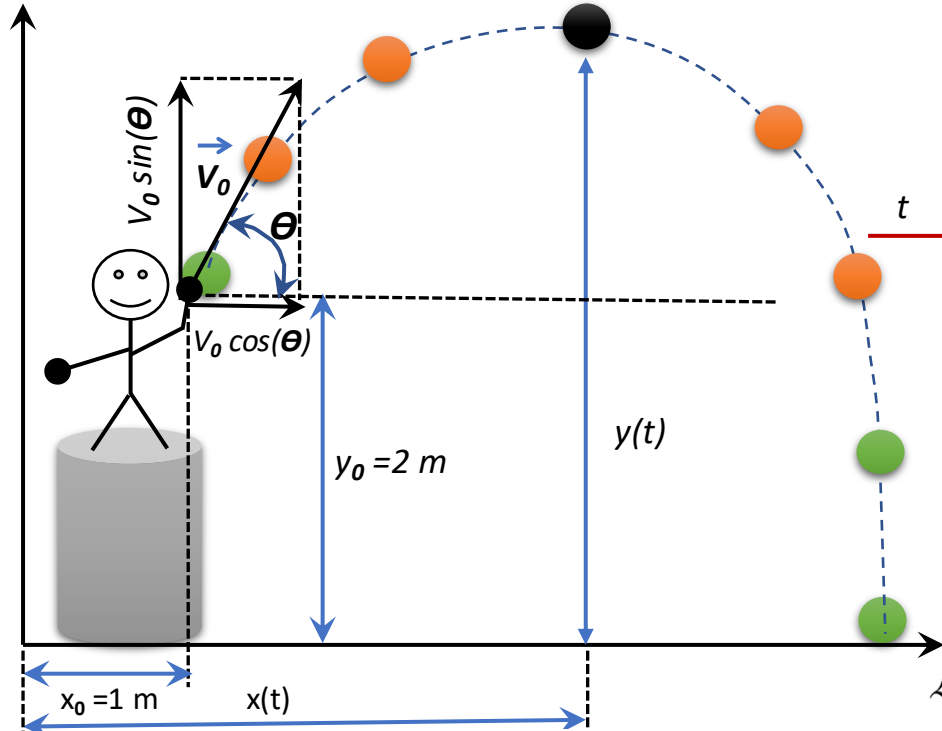
```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
plt.plot(t_test, x, label="x_exact", linewidth=2)
plt.plot(t_test, y, label="y Exact", linewidth=2)
plt.scatter(t_test, x, color="red", label="X data points")
plt.scatter(t_test, y, color="blue", label="Y data points")
plt.xlabel("t")
plt.ylabel("x(t), y(t)")
plt.title("X and Y position")
plt.legend()
plt.grid()
```

```
plt.subplot(1, 2, 2)
plt.plot(x, y, label="Trajectory", linewidth=2)
plt.scatter(x, y, color="red", label="X data points")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Trajectory")
plt.legend()
plt.grid()
```



Data Driven Model : Neural Network



Hidden layer 64 neurones

Hidden layer 64 neurones

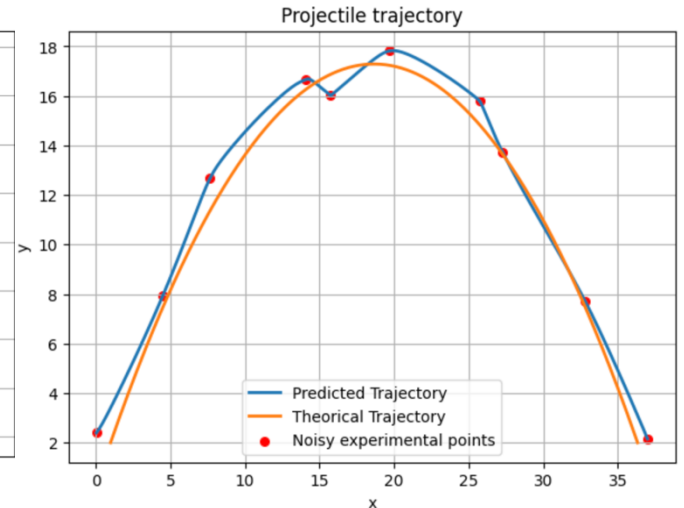
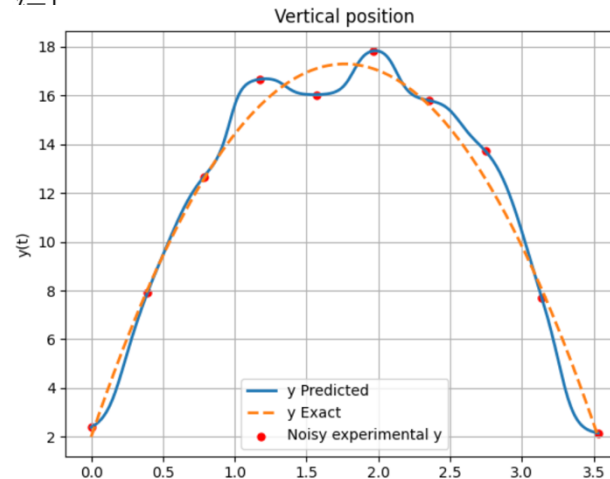
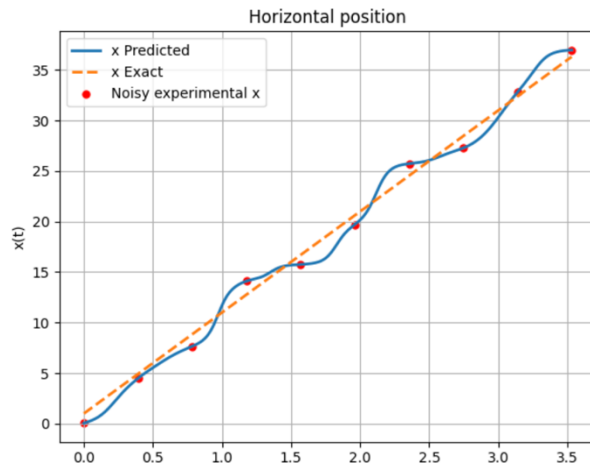
$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

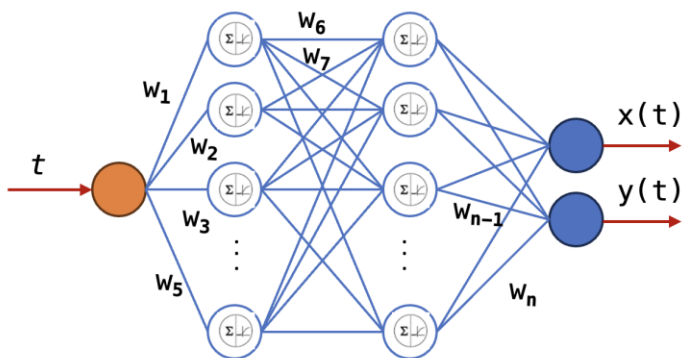
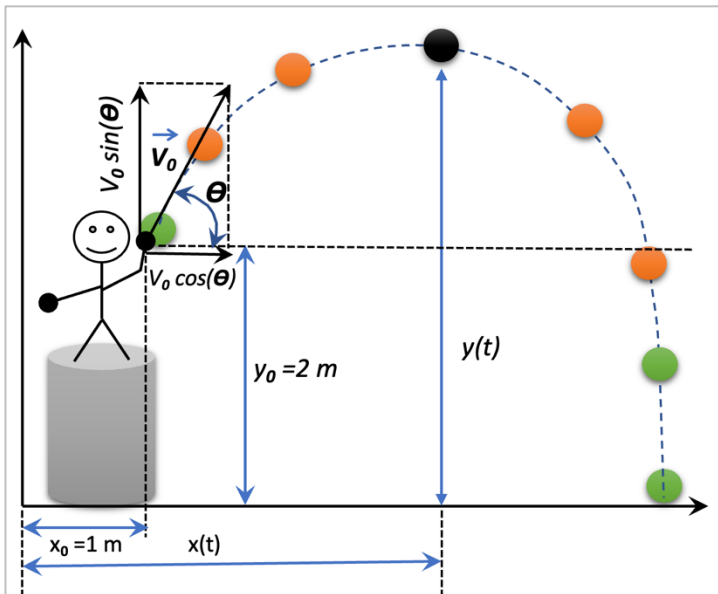
$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N [(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2]$$

$$\begin{cases} w_i^{(k+1)} = w_i^{(k)} - \eta \frac{\partial \mathcal{L}}{\partial w_i^{(k)}} & \text{pour } i = 1, 2, 3 \\ b^{(k+1)} = b^{(k)} - \eta \frac{\partial \mathcal{L}}{\partial b^{(k)}} \end{cases} \quad \mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

η : taux d'apprentissage (learning rate)

	time	X	y
0	0.000000	0.471534	2.219682
1	0.588532	9.586019	15.523645
2	1.177065	17.993155	25.836216
3	1.765597	28.226082	33.179218
4	2.354130	35.791260	35.383904
5	2.942662	44.638611	36.379066
6	3.531194	54.629513	32.692162
7	4.119727	62.136314	26.421669
8	4.708259	71.856407	15.405033
9	5.296792	80.820435	2.086739





$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N [(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2]$$

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

time = torch.tensor([
    0.0000,
    0.3924,
    0.7847,
    1.1771,
    1.5694,
    1.9618,
    2.3541,
    2.7465,
    3.1388,
    3.5312])

pos_exp = torch.tensor([
    0.0488, 2.3954,
    4.4880, 7.9184,
    7.6540, 12.6630,
    14.1065, 16.6683,
    15.7570, 16.0307,
    19.7154, 17.8220,
    25.7322, 15.7916,
    27.2776, 13.7168,
    32.8069, 7.7049,
    36.9753, 2.1562])
```

Model

```
data_model = nn.Sequential(
    nn.Linear(1, 64),
    nn.Tanh(),
    nn.Linear(64, 2) # sortie : [x(t), y(t)]
)
```

Algo qui permet d'optimiser les paramètres du modèle

```
Optimizer = optim.Adam(data_model.parameters(), lr=0.001)
critetion = nn.MSELoss() # Mean Squared Error Loss
```

```
epochs = 8000
```

```
for epoch in range(epochs):
```

```
    model.train() # Positionner le modèle en mode train
```

```
    pred = data_model(time) # Prédiction
```

```
    x_pred = pred[:, 0:1] # Première colonne (x)
```

```
    y_pred = pred[:, 1:2] # Deuxième colonne (y)
```

```
    # Calculer le LOSS : Mean Squared Error
```

```
    loss_data = torch.mean( (pred - pos_exp) ** 2 )
```

```
    # loss_data = critetion(pred, position)
```

```
    optimizer.zero_grad() # Init gradient des params
```

```
    loss_data.backward() # d(loss)/dw et d(loss)/db
```

```
    optimizer.step() # Update weights : w = w - lr * dl/dw
```

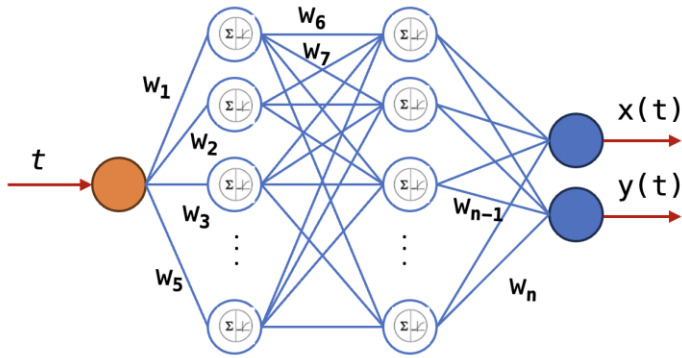
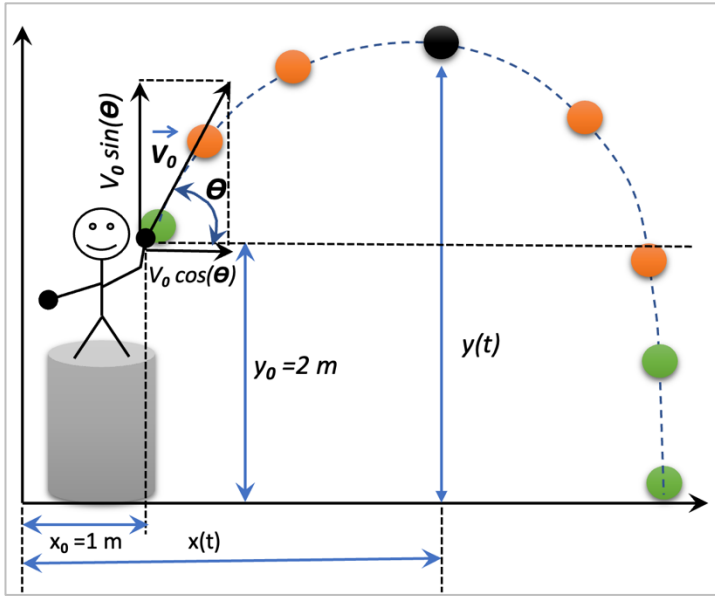
```
if (epoch + 1) % 1000 == 0:
```

```
    print(
```

```
        f"Epoch [{epoch+1}/{epochs}] "
```

```
        f"Loss: {loss_data.item():.6f} | "
```

```
)
```



$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N [(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2]$$

Évaluation

t_final = 3.5312

t_test = torch.linspace(0, t_final, 300).view(-1, 1)

model.eval() # Positionner le modèle en mode Evaluation

with torch.no_grad():

pred_test = data_model(t_test)

x_pred = pred_test[:, 0:1]

y_pred = pred_test[:, 1:2]

Solution théorique exacte

x_exact = x0 + vx0 * t_test

y_exact = y0 + vy0 * t_test - 0.5 * g * t_test ** 2

plt.subplot(1, 2, 2)

plt.plot(t_test, y_pred, label="y Predicted")

plt.plot(t_test, y_exact, "--", label="y Exact")

plt.scatter(time, pos_exp[:,1:2], color="red", s=25,label="Noisy exp y")

plt.xlabel("t")

plt.ylabel("y(t)")

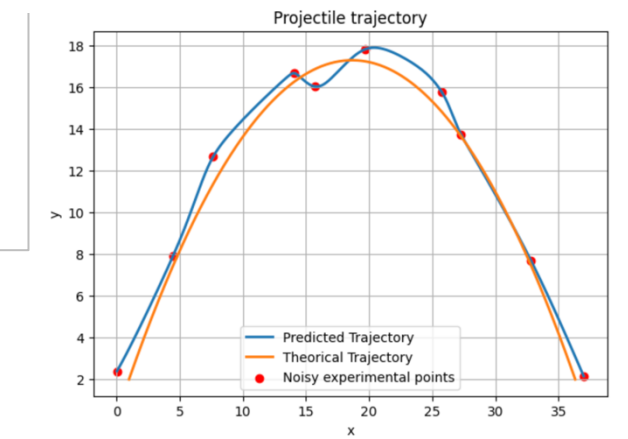
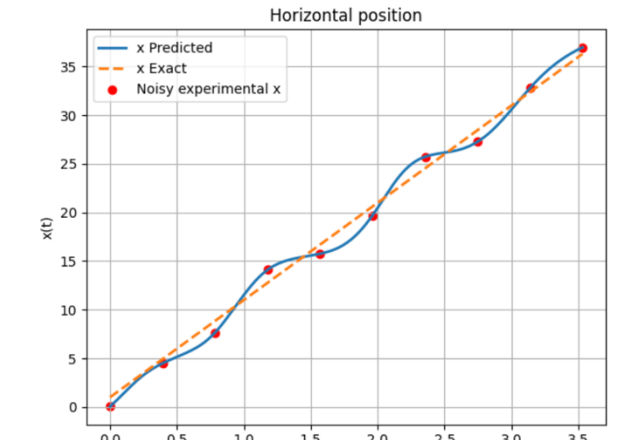
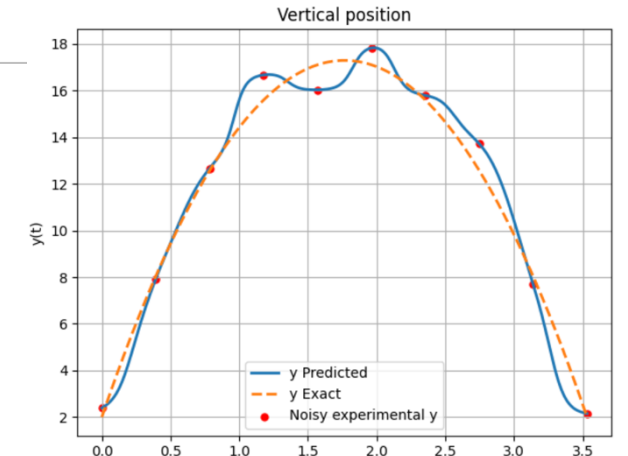
plt.title("Vertical position")

plt.legend()

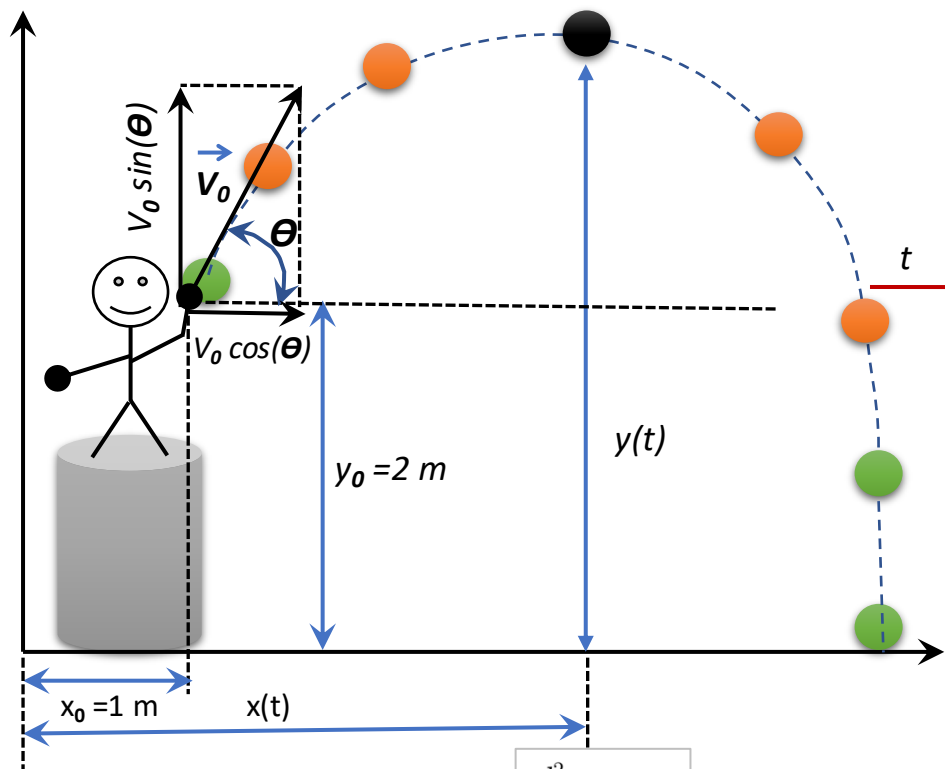
plt.grid()

plt.tight_layout()

plt.show()



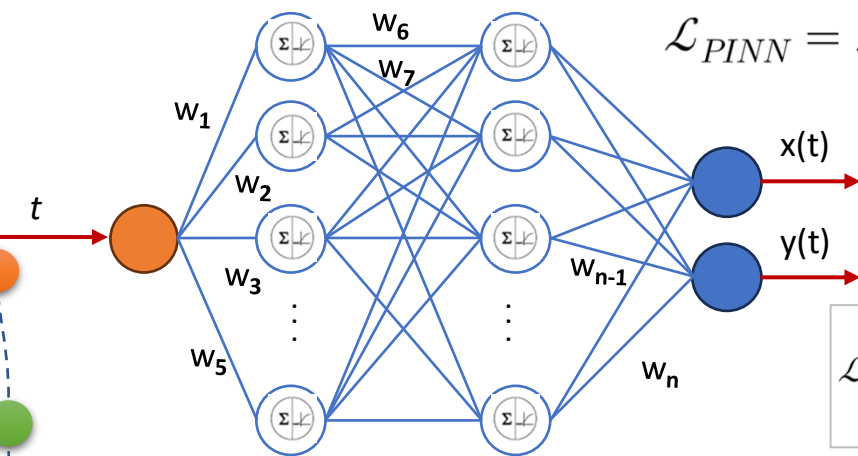
Physics Informed Neural Network



	time	X	y
0	0.000000	0.471534	2.219682
1	0.588532	9.586019	15.523645
2	1.177065	17.993155	25.836216
3	1.765597	28.226082	33.179218
4	2.354130	35.791260	35.383904
5	2.942662	44.638611	36.379066
6	3.531194	54.629513	32.692162
7	4.119727	62.136314	26.421669
8	4.708259	71.856407	15.405033
9	5.296792	80.820435	2.086739

$$\frac{d^2 x}{dt^2} = 0$$
$$\frac{d^2 y}{dt^2} = -g$$

$$x(0) = x_0, \quad y(0) = y_0$$
$$\frac{dx}{dt}(0) = v_0 \cos \theta$$
$$\frac{dy}{dt}(0) = v_0 \sin \theta$$

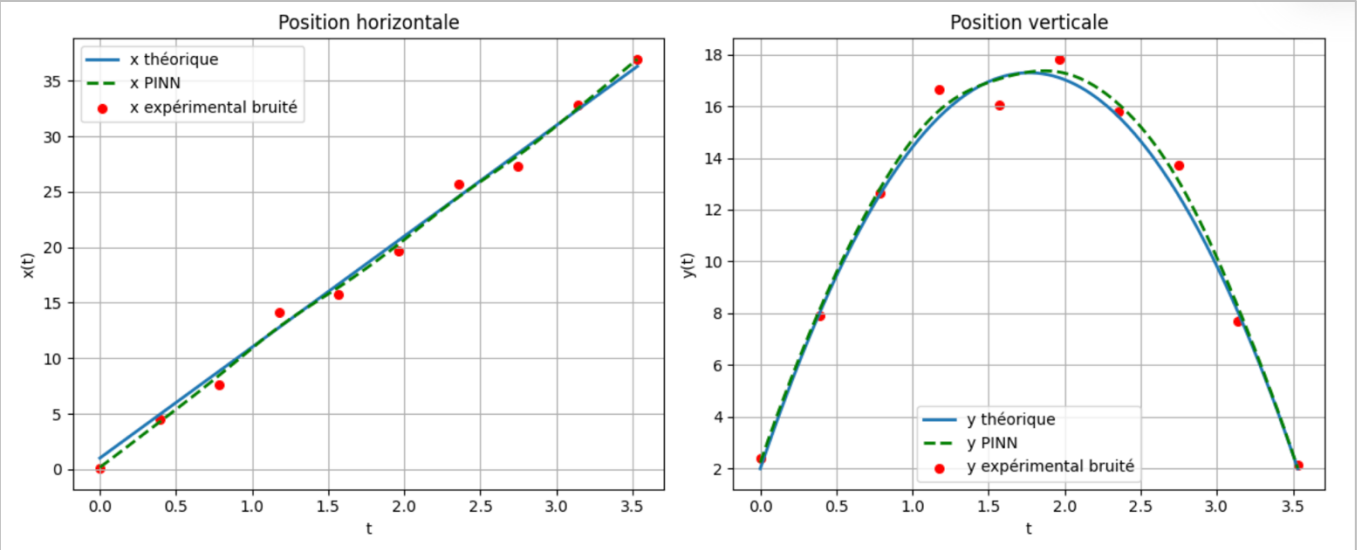


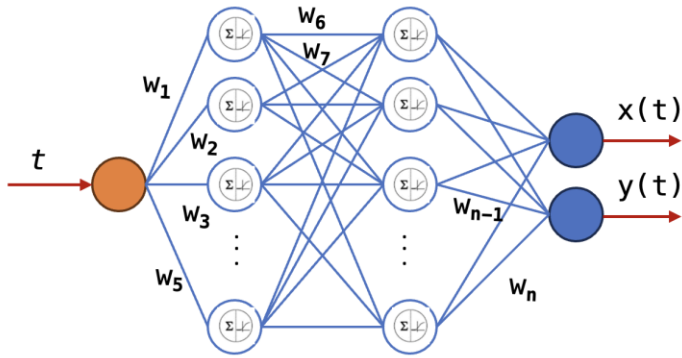
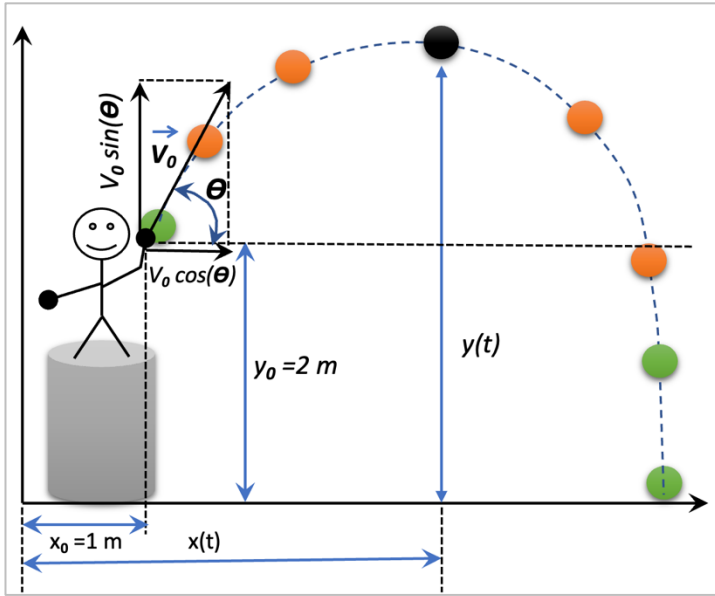
$$\mathcal{L}_{PINN} = \lambda_{data} \mathcal{L}_{data} + \lambda_{physics} \mathcal{L}_{physics} + \lambda_{IC} \mathcal{L}_{IC}$$

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N [(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2]$$

$$\mathcal{L}_{physics} = \frac{1}{N_f} \sum_{i=1}^{N_f} \left[\left(\frac{d^2 \hat{x}(t_i)}{dt^2} \right)^2 + \left(\frac{d^2 \hat{y}(t_i)}{dt^2} + g \right)^2 \right]$$

$$\mathcal{L}_{IC} = (\hat{x}(0) - x_0)^2 + (\hat{y}(0) - y_0)^2 + \left(\frac{d\hat{x}}{dt}(0) - v_0 \cos \theta \right)^2 + \left(\frac{d\hat{y}}{dt}(0) - v_0 \sin \theta \right)^2$$





$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N [(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2]$$

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import math
```

```
# =====
# 1. Configuration
# =====
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
torch.manual_seed(42)
```

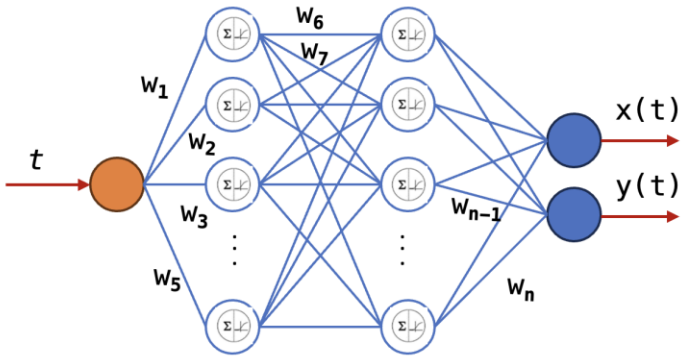
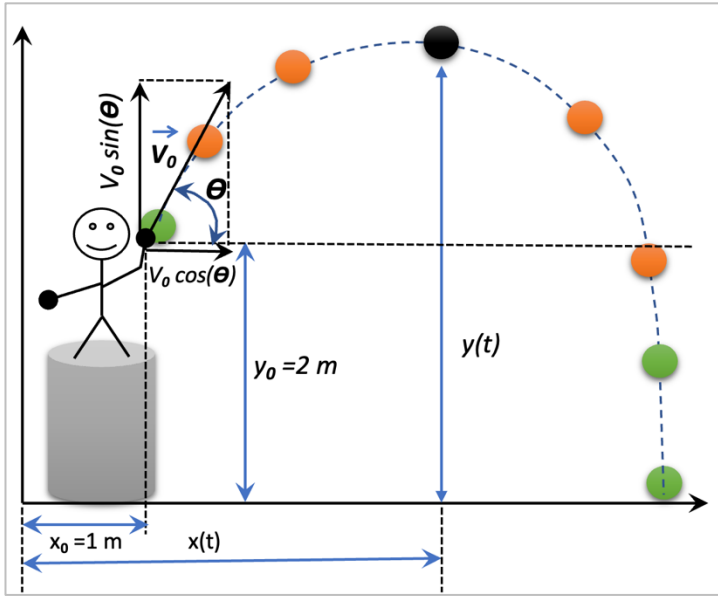
```
# Paramètres physiques
g = 9.81
```

```
# Conditions initiales
x0 = 1.0
y0 = 2.0
```

```
teta = math.pi/3
v0 = 20 # m/s
vx0 = v0*math.cos(teta)
vy0 = v0*math.sin(teta)
```

```
# Temps de vol théorique pour y = 2
T_final = 2 * vy0 / g
```

```
print("Device :", device)
print("Temps de vol théorique :", T_final)
```



$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N [(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2]$$

```
# =====
# 2. Helper pour les dérivées
# =====
```

```
def derivative(y, t):
```

```
    """
```

```
    Calcule dy/dt avec autograd.
```

```
    """
```

```
    return torch.autograd.grad(
```

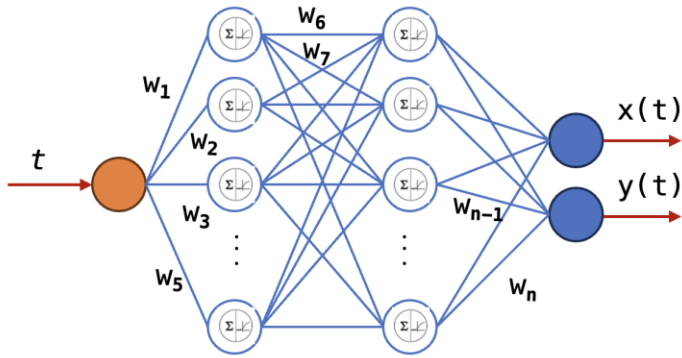
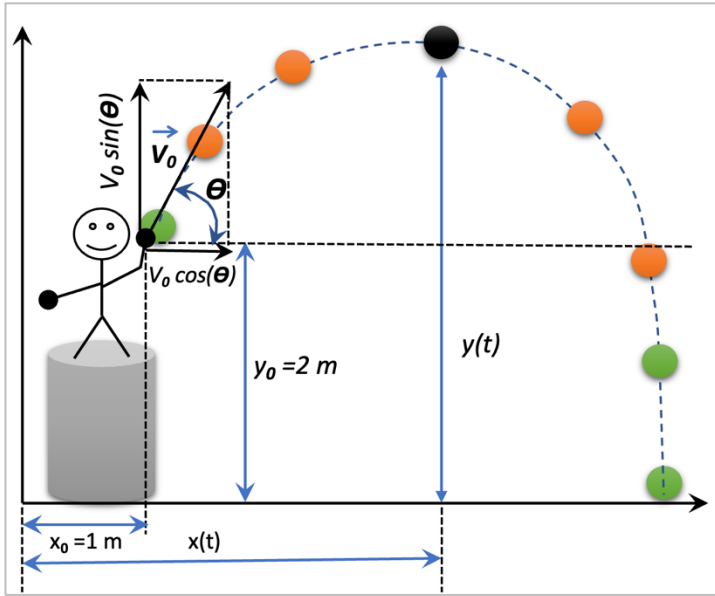
```
        y,
```

```
        t,
```

```
        grad_outputs=torch.ones_like(y),
```

```
        create_graph=True
```

```
    )[0]
```



$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N [(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2]$$

```
model = nn.Sequential(
    nn.Linear(1, 64),
    nn.Tanh(),

    nn.Linear(64, 64),
    nn.Tanh(),

    nn.Linear(64, 64),
    nn.Tanh(),

    nn.Linear(64, 2)
).to(device)
```

```
# =====
# 3. Réseau PINN
# =====
```

```
class ProjectilePINN(nn.Module):
    def __init__(self):
        super().__init__()
```

```
        self.net = nn.Sequential(
            nn.Linear(1, 64),
            nn.Tanh(),

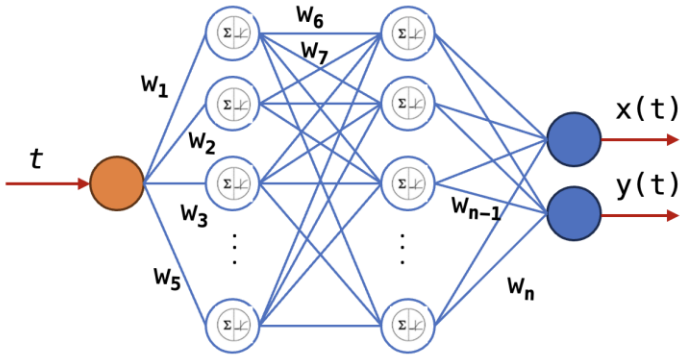
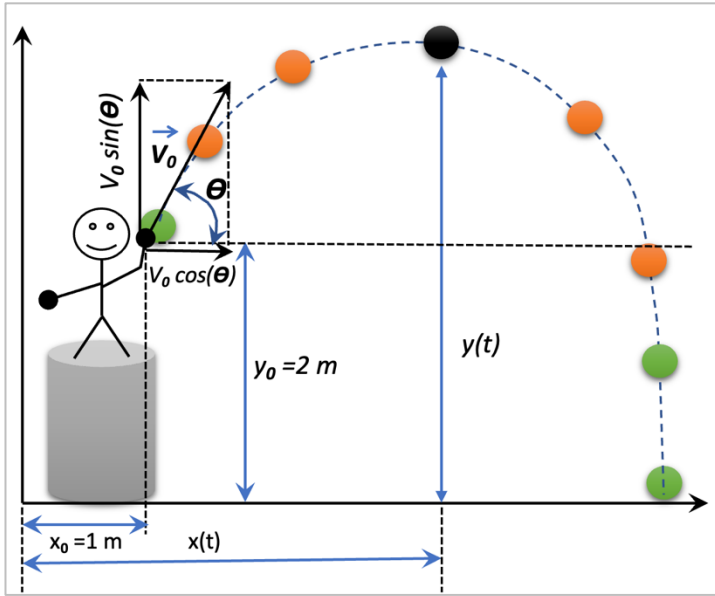
            nn.Linear(64, 64),
            nn.Tanh(),

            nn.Linear(64, 64),
            nn.Tanh(),

            nn.Linear(64, 2)
        )
```

```
    def forward(self, t):
        return self.net(t)
```

```
model = ProjectilePINN().to(device)
```



$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N [(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2]$$

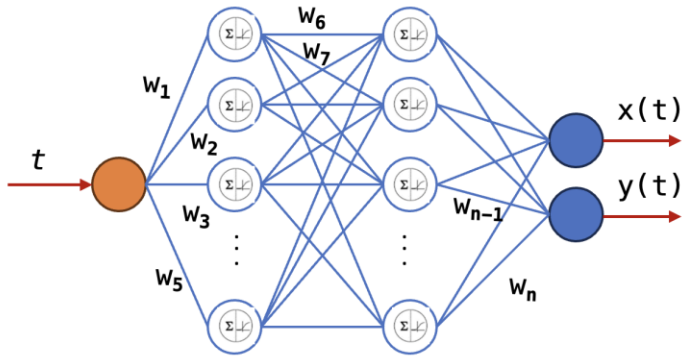
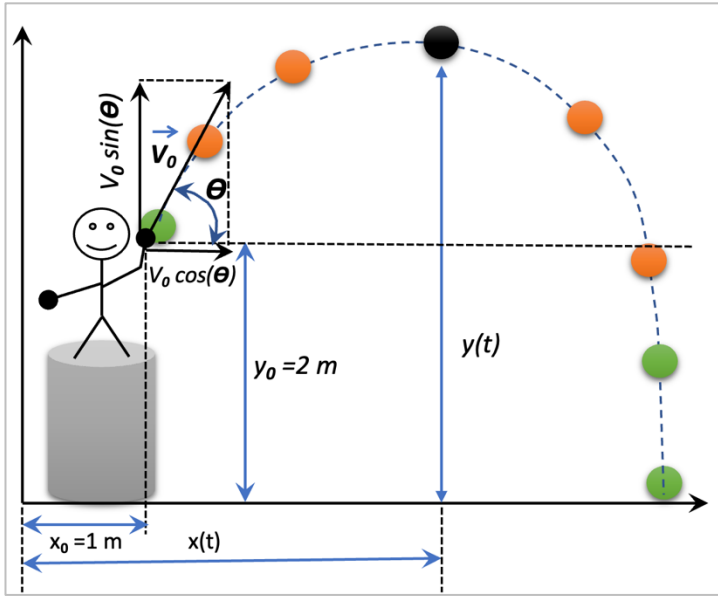
```
# =====
# 4. Points de collocation physique
# =====
```

```
N_f = 200
```

```
t_f = torch.linspace(0, T_final, N_f).view(-1, 1).to(device)
t_f.requires_grad_(True)
```

```
# =====
# 5. Condition initiale t = 0
# =====
```

```
t0 = torch.tensor([[0.0]], dtype=torch.float32).to(device)
t0.requires_grad_(True)
```



$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N [(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2]$$

```
# =====
# 6. Données expérimentales simulées
# =====
```

```
N_data = 10
noise_level = 0.9
```

```
t_data = torch.linspace(0, T_final, N_data).view(-1, 1).to(device)
```

```
# Solution théorique aux temps expérimentaux
```

```
x_data_exact = x0 + vx0 * t_data
```

```
y_data_exact = y0 + vy0 * t_data - 0.5 * g * t_data ** 2
```

```
# Bruit gaussien
```

```
noise_x = noise_level * torch.randn_like(x_data_exact)
```

```
noise_y = noise_level * torch.randn_like(y_data_exact)
```

```
# Données expérimentales bruitées
```

```
x_data_exp = x_data_exact + noise_x
```

```
y_data_exp = y_data_exact + noise_y
```

```
# Regrouper en un tenseur de shape : [N_data, 2]
```

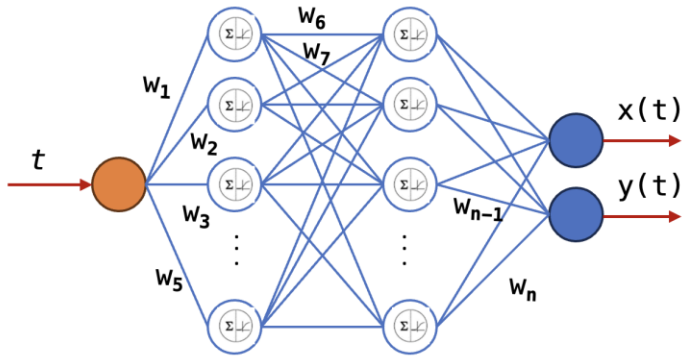
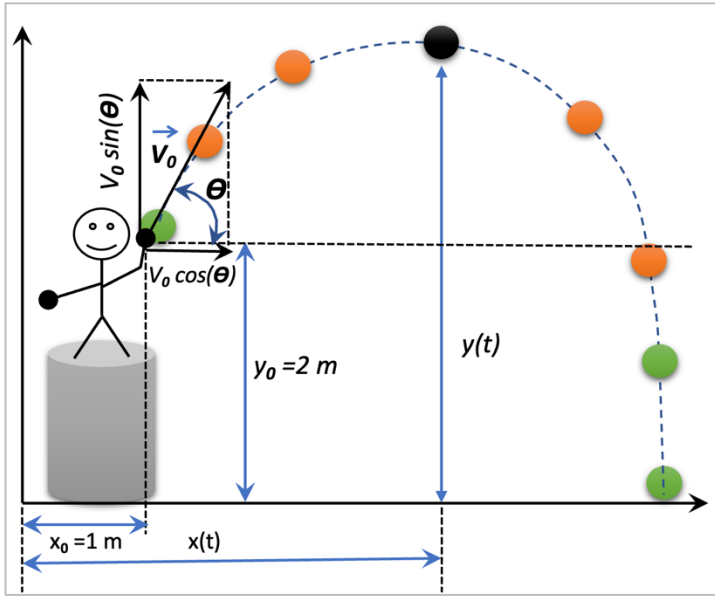
```
data_exp = torch.cat([x_data_exp, y_data_exp], dim=1)
```

$$x(t) = x_0 + v_0 \cos(\theta) t$$

$$y(t) = y_0 + v_0 \sin(\theta) t - \frac{1}{2} g t^2$$

time
0.000000
0.588532
1.177065
1.765597
2.354130
2.942662
3.531194
4.119727
4.708259
5.296792

x	y
0.471534	2.219682
9.586019	15.523645
17.993155	25.836216
28.226082	33.179218
35.791260	35.383904
44.638611	36.379066
54.629513	32.692162
62.136314	26.421669
71.856407	15.405033
80.820435	2.086739



$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N \left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2 \right]$$

```
# =====
# 7. Optimiseur
# =====
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

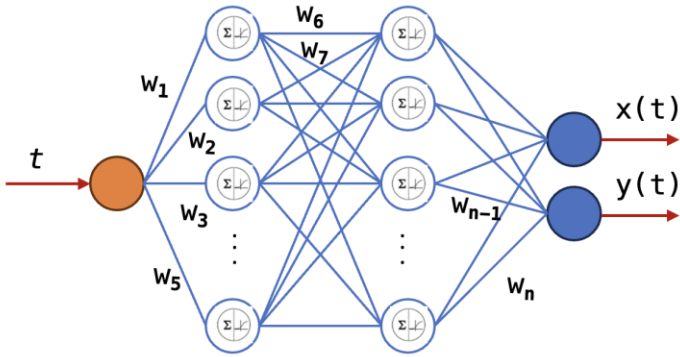
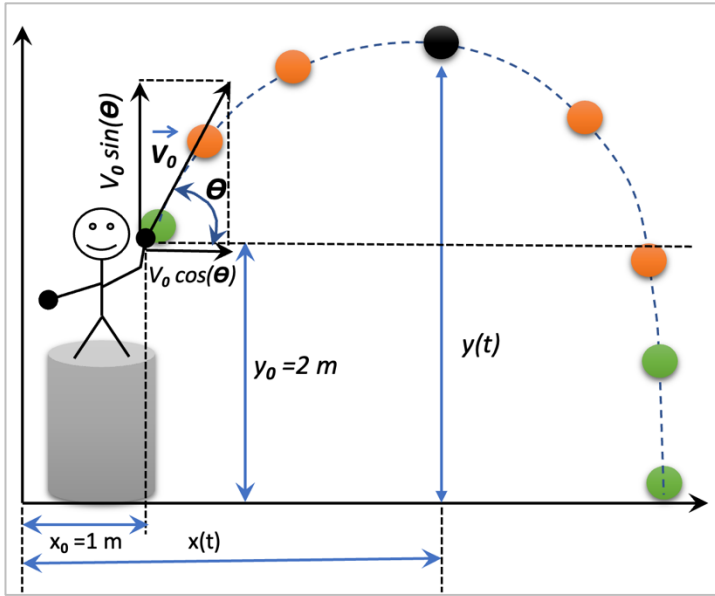
```
# Poids des différentes losses
```

```
lambda_physics = 1
```

```
lambda_ic = 1
```

```
lambda_data = 300
```

$$\mathcal{L}_{PINN} = \lambda_{data} \mathcal{L}_{data} + \lambda_{physics} \mathcal{L}_{physics} + \lambda_{IC} \mathcal{L}_{IC}$$



$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N [(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2]$$

```
# =====
# 8. Entraînement
# =====
epochs = 8000
for epoch in range(epochs):
    model.train()
    # -----
    # Loss physique
    # -----
    pred_f = model(t_f)
    x_pred = pred_f[:, 0:1]
    y_pred = pred_f[:, 1:2]

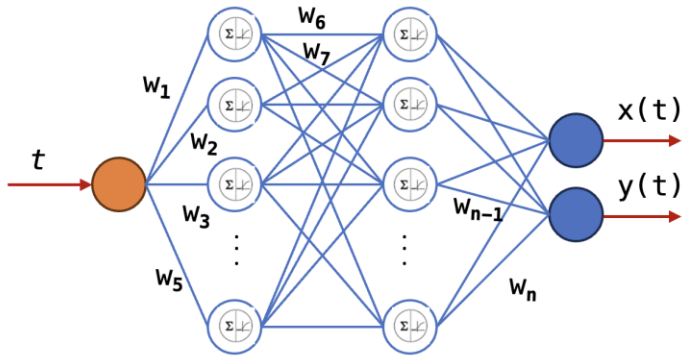
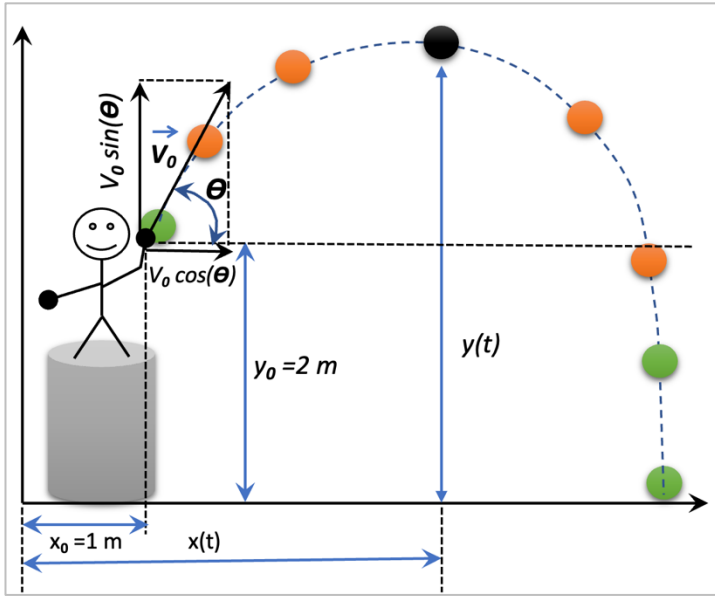
    # Dérivées premières
    x_t = derivative(x_pred, t_f)
    y_t = derivative(y_pred, t_f)

    # Dérivées secondes
    x_tt = derivative(x_t, t_f)
    y_tt = derivative(y_t, t_f)

    # Equations physiques :
    # x''(t) = 0
    # y''(t) = -g => y''(t) + g = 0
    residual_x = x_tt
    residual_y = y_tt + g

    loss_physics = torch.mean(residual_x ** 2) + torch.mean(residual_y ** 2)
```

$$\mathcal{L}_{physics} = \frac{1}{N_f} \sum_{i=1}^{N_f} \left[\left(\frac{d^2 \hat{x}(t_i)}{dt^2} \right)^2 + \left(\frac{d^2 \hat{y}(t_i)}{dt^2} + g \right)^2 \right]$$



$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N [(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2]$$

```
# -----
# Loss conditions initiales
# -----
```

```
pred0 = model(t0)
```

```
x0_pred = pred0[:, 0:1]
y0_pred = pred0[:, 1:2]
```

```
x0_t = derivative(x0_pred, t0)
y0_t = derivative(y0_pred, t0)
```

```
loss_ic_position = (x0_pred - x0) ** 2 + (y0_pred - y0) ** 2
loss_ic_velocity = (x0_t - vx0) ** 2 + (y0_t - vy0) ** 2
```

```
loss_ic = torch.mean(loss_ic_position + loss_ic_velocity)
```

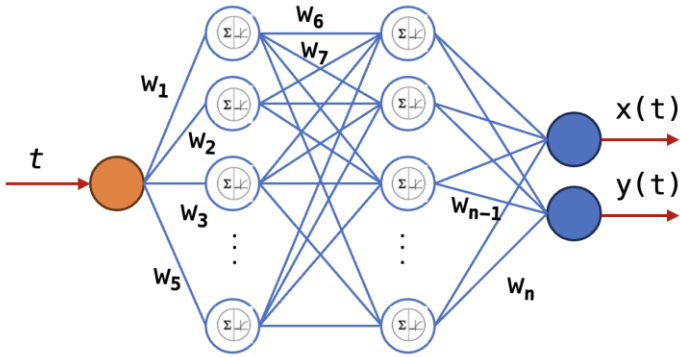
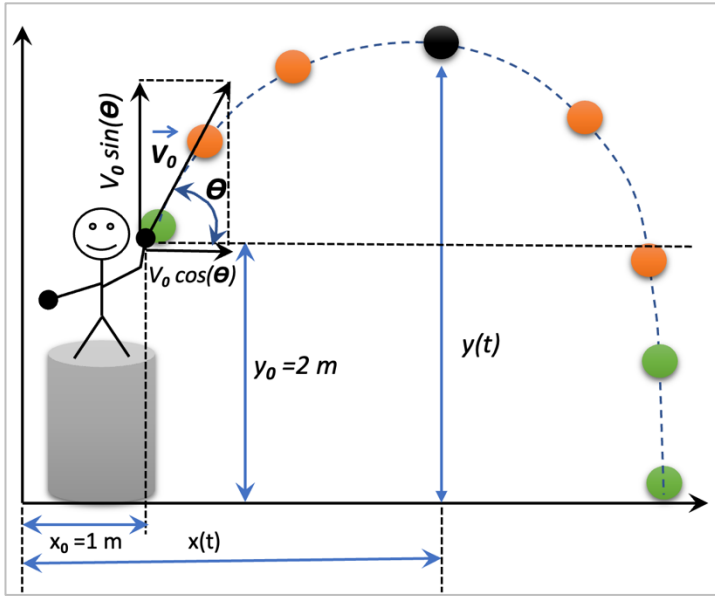
```
# -----
# Loss données expérimentales
# -----
```

```
pred_data = model(t_data)
```

```
loss_data = torch.mean((pred_data - data_exp) ** 2)
```

$$\mathcal{L}_{IC} = (\hat{x}(0) - x_0)^2 + (\hat{y}(0) - y_0)^2 + \left(\frac{d\hat{x}}{dt}(0) - v_0 \cos \theta \right)^2 + \left(\frac{d\hat{y}}{dt}(0) - v_0 \sin \theta \right)^2$$

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N [(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2]$$



$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N \left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2 \right]$$

```
# -----
# Loss totale
# -----

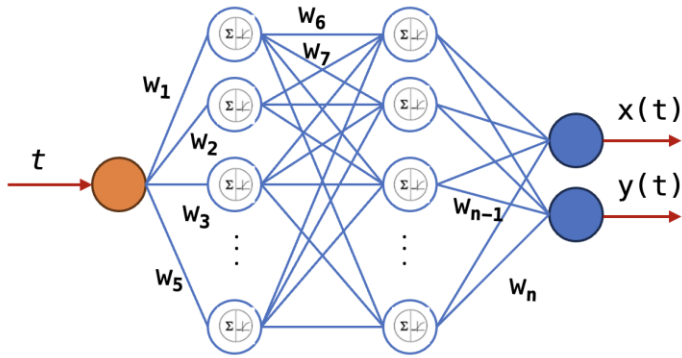
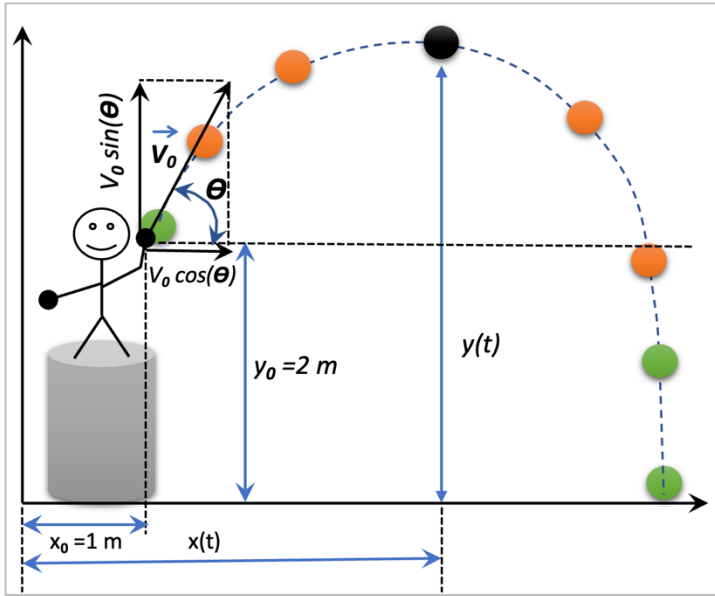
loss = (
    lambda_physics * loss_physics
    + lambda_ic * loss_ic
    + lambda_data * loss_data
)

# -----
# Backpropagation
# -----

optimizer.zero_grad()
loss.backward()
optimizer.step()

if (epoch + 1) % 1000 == 0:
    print(
        f"Epoch [{epoch+1}/{epochs}] "
        f"Loss: {loss.item():.6f} | "
        f"Physics: {loss_physics.item():.6f} | "
        f"IC: {loss_ic.item():.6f} | "
        f>Data: {loss_data.item():.6f}"
    )
```

$$\mathcal{L}_{PINN} = \lambda_{data} \mathcal{L}_{data} + \lambda_{physics} \mathcal{L}_{physics} + \lambda_{IC} \mathcal{L}_{IC}$$



$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N [(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2]$$

```
# =====
# 9. Évaluation
# =====

model.eval()

t_test = torch.linspace(0, T_final, 300).view(-1, 1).to(device)

with torch.no_grad():
    pred_test = model(t_test)

x_pinn = pred_test[:, 0:1]
y_pinn = pred_test[:, 1:2]

# Solution théorique exacte
x_exact = x0 + vx0 * t_test
y_exact = y0 + vy0 * t_test - 0.5 * g * t_test ** 2

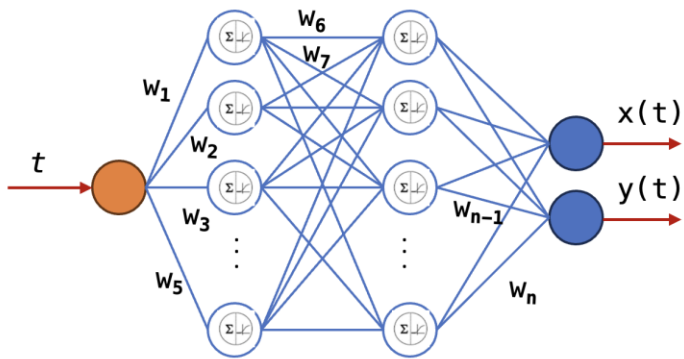
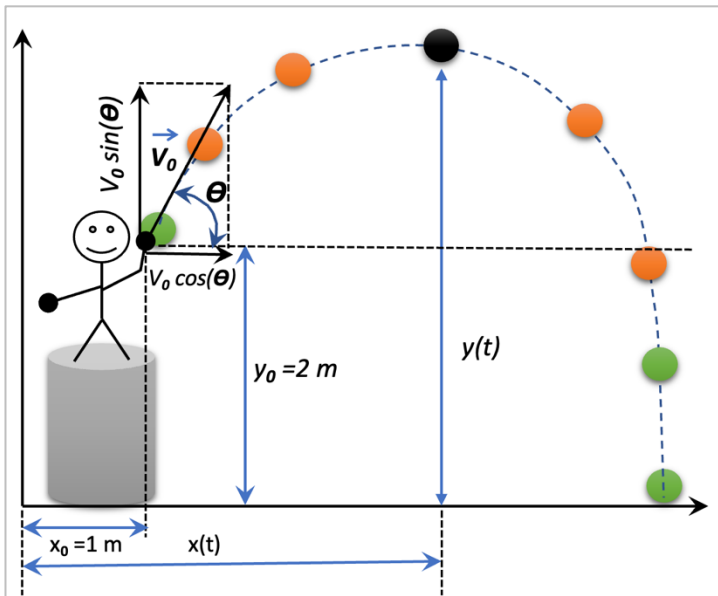
# =====
# 10. Conversion CPU/Numpy
# =====

t_np = t_test.cpu().numpy()

x_pinn_np = x_pinn.cpu().numpy()
y_pinn_np = y_pinn.cpu().numpy()

x_exact_np = x_exact.cpu().numpy()
y_exact_np = y_exact.cpu().numpy()

t_data_np = t_data.cpu().numpy()
x_data_exp_np = x_data_exp.cpu().numpy()
y_data_exp_np = y_data_exp.cpu().numpy()
```



$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

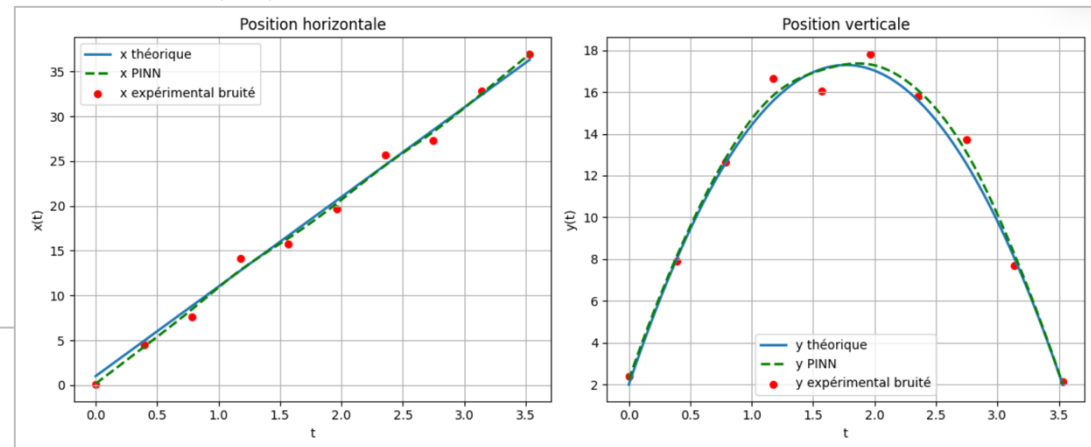
$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N [(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2]$$

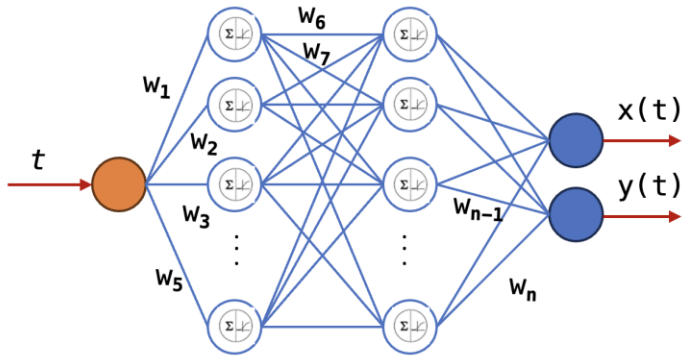
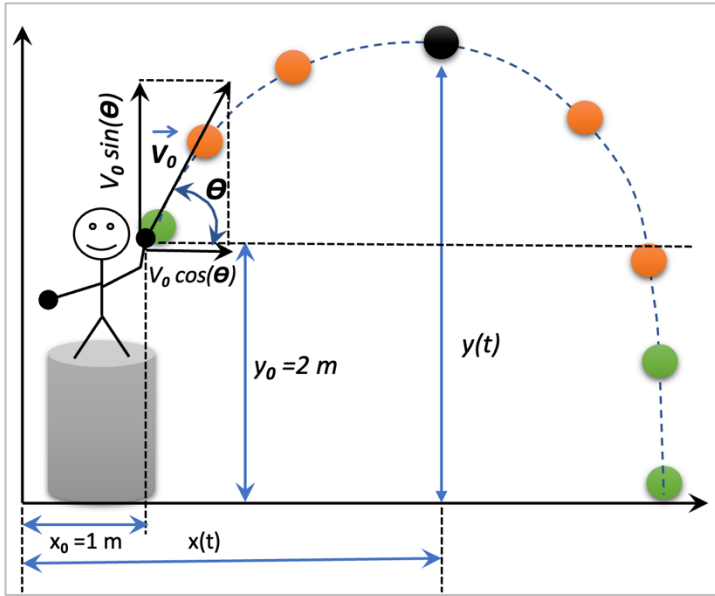
```
# =====
# 11. Visualisation x(t) et y(t)
# =====
```

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
plt.plot(t_np, x_exact_np, label="x théorique", linewidth=2)
plt.plot(t_np, x_pinn_np, "--", color="green", label="x PINN", linewidth=2)
plt.scatter(t_data_np, x_data_exp_np, color="red", s=30, label="x expérimental bruité")
plt.xlabel("t")
plt.ylabel("x(t)")
plt.title("Position horizontale")
plt.legend()
plt.grid()
```

```
plt.subplot(1, 2, 2)
plt.plot(t_np, y_exact_np, label="y théorique", linewidth=2)
plt.plot(t_np, y_pinn_np, "--", color="green", label="y PINN", linewidth=2)
plt.scatter(t_data_np, y_data_exp_np, color="red", s=30, label="y expérimental bruité")
plt.xlabel("t")
plt.ylabel("y(t)")
plt.title("Position verticale")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```





$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N [(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2]$$

```
# =====
# 12. Visualisation trajectoire y(x)
# =====
```

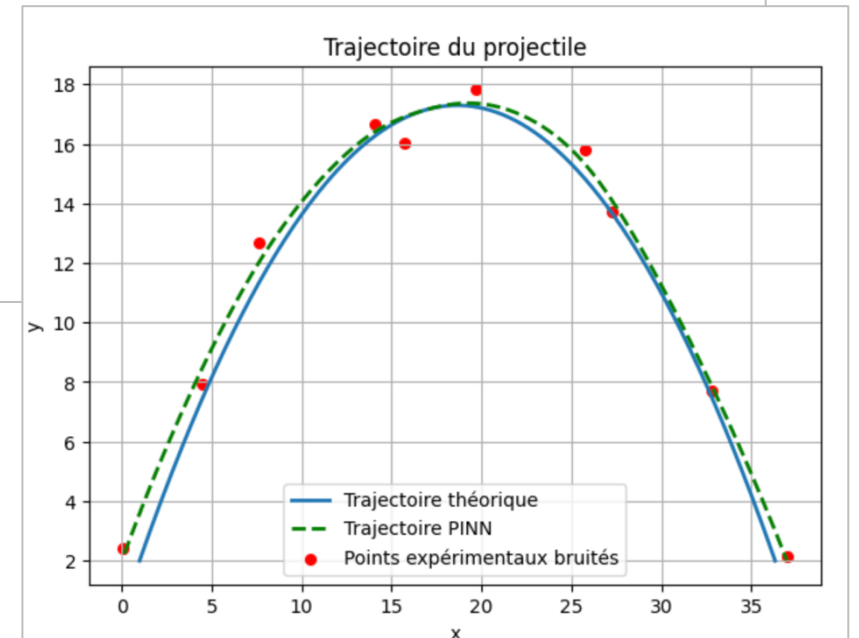
```
plt.figure(figsize=(7, 5))
```

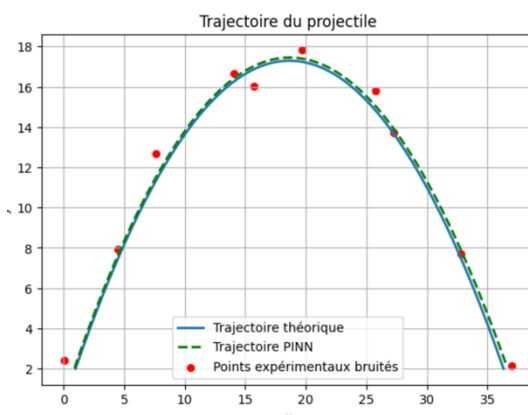
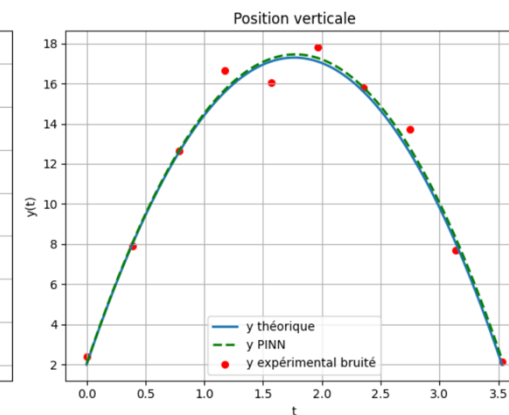
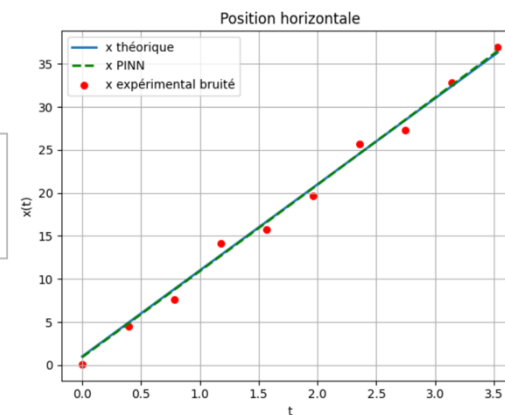
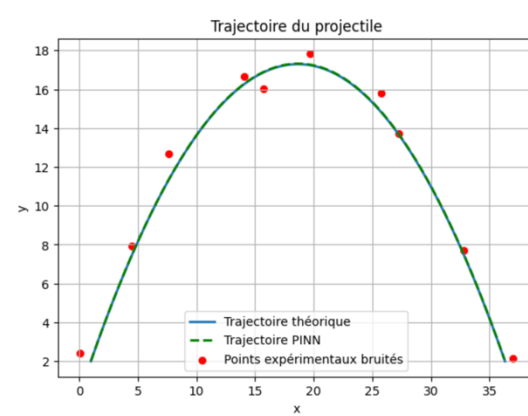
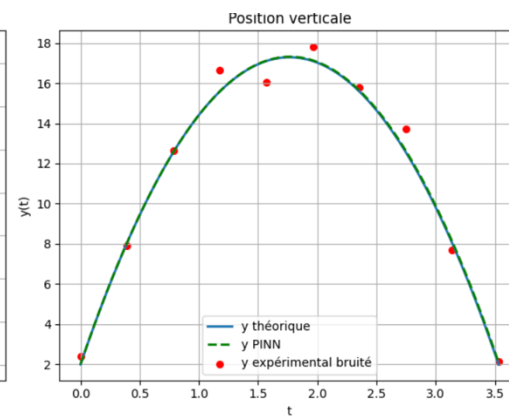
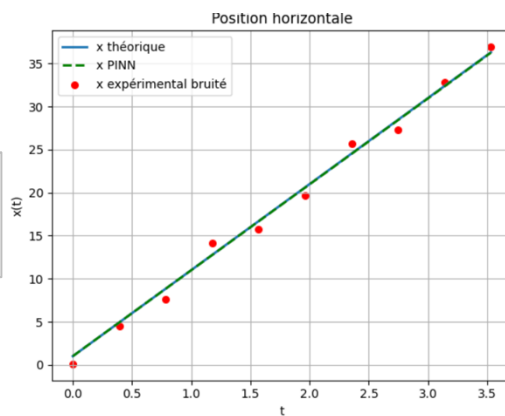
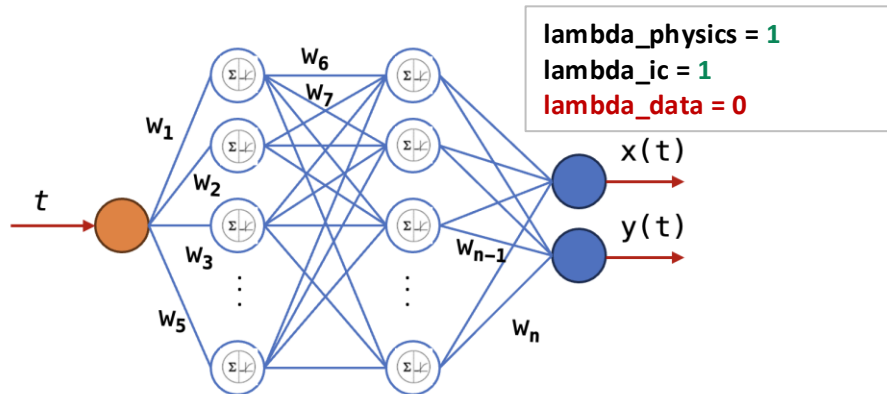
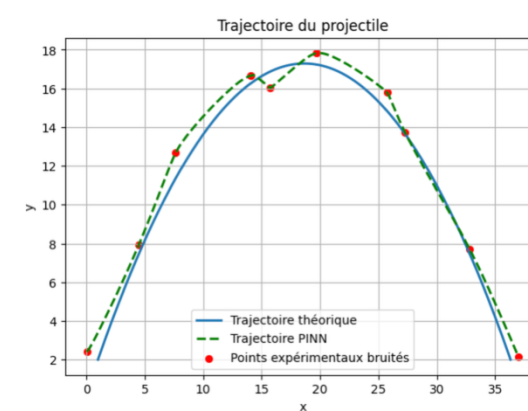
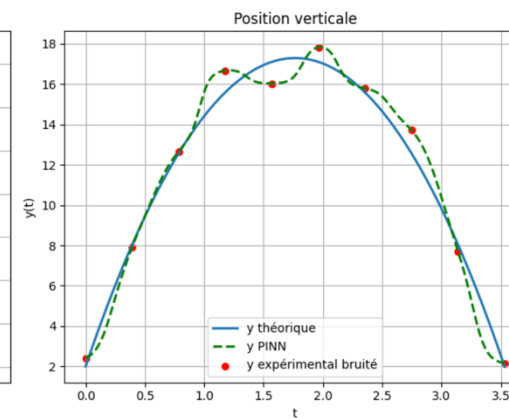
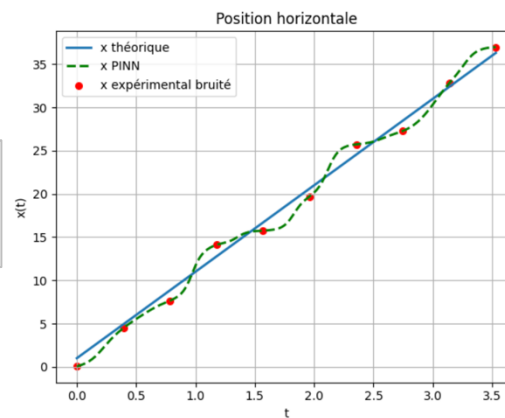
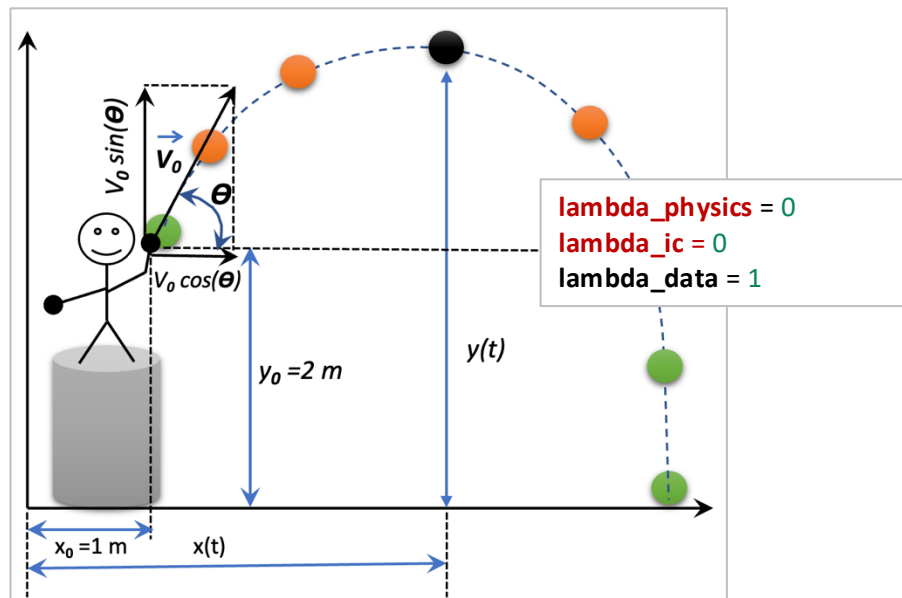
```
plt.plot( x_exact_np, y_exact_np, label="Trajectoire théorique",linewidth=2)
```

```
plt.plot(x_pinn_np, y_pinn_np, "--", color="green", label="Trajectoire PINN",
        linewidth=2
    )
```

```
plt.scatter( x_data_exp_np, y_data_exp_np, color="red", s=30,
            label="Points expérimentaux bruités"
        )
```

```
plt.xlabel("x")
plt.ylabel("y")
plt.title("Trajectoire du projectile")
plt.legend()
plt.grid()
plt.show()
```

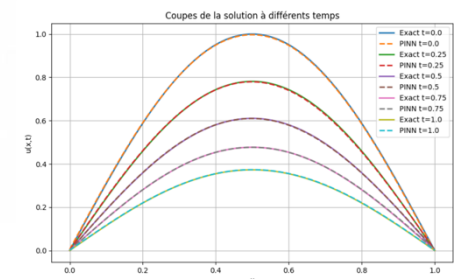
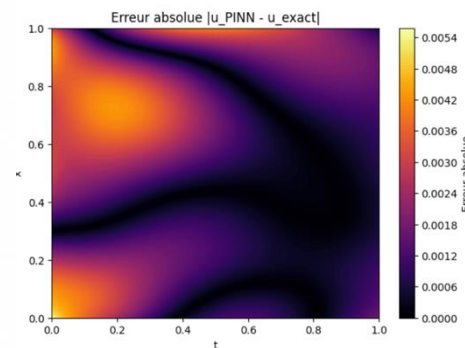
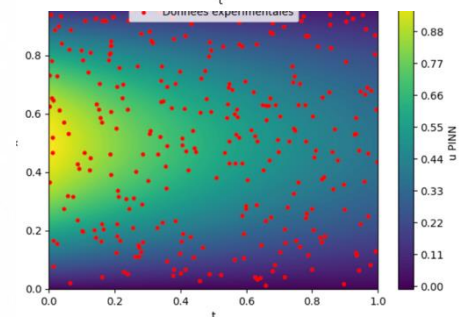
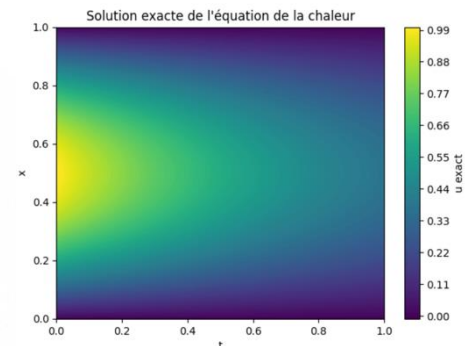
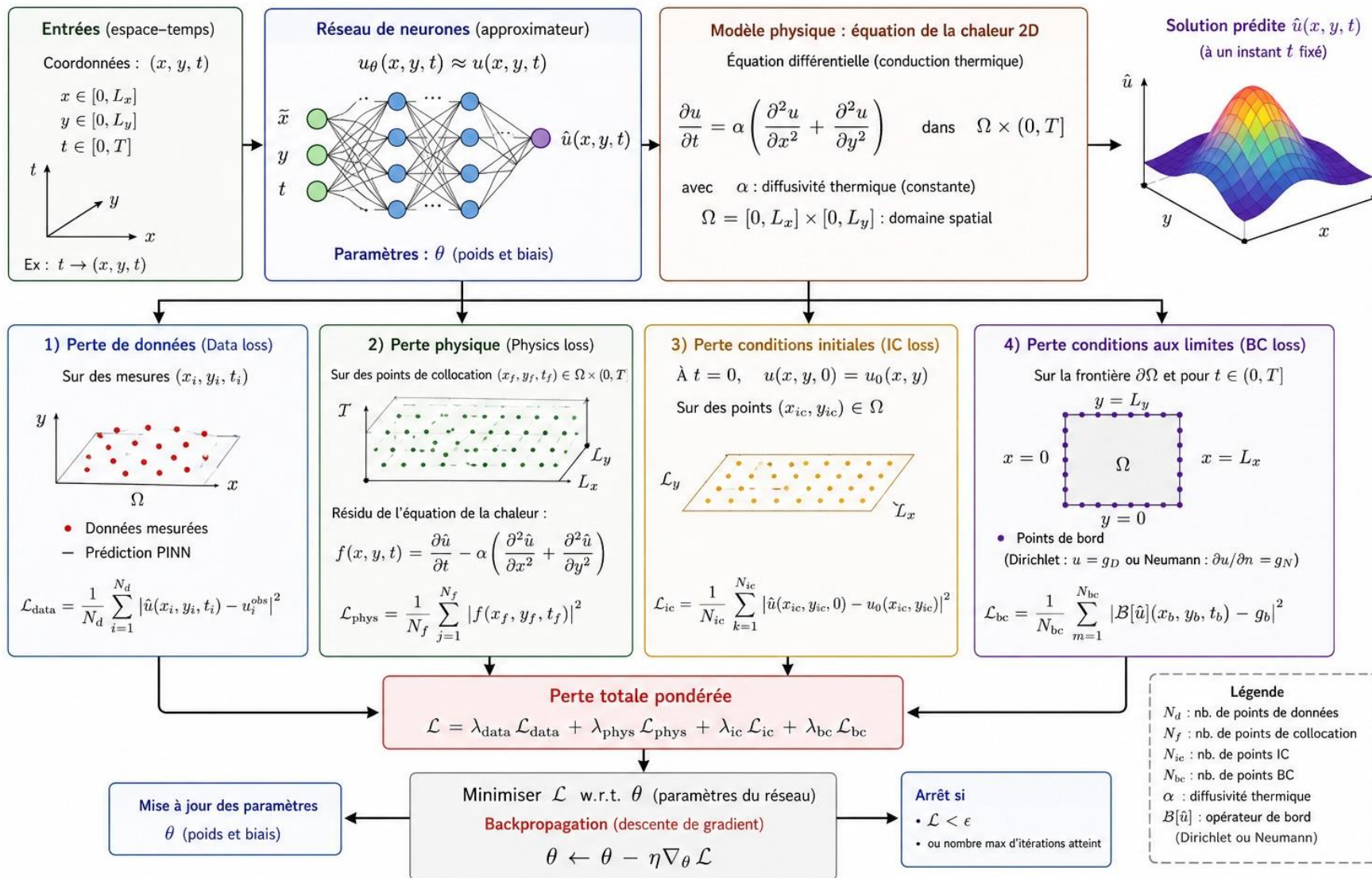




$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

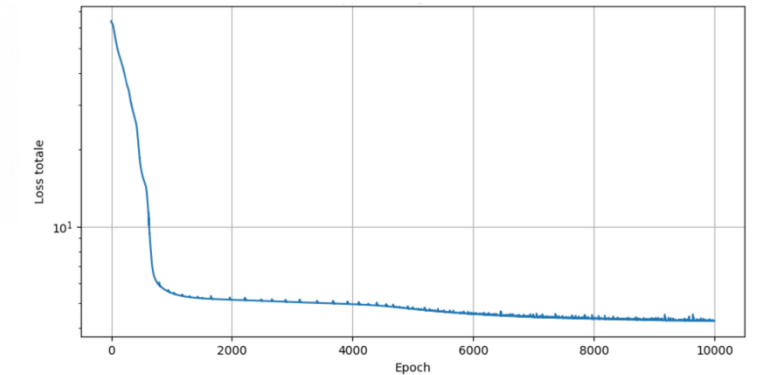
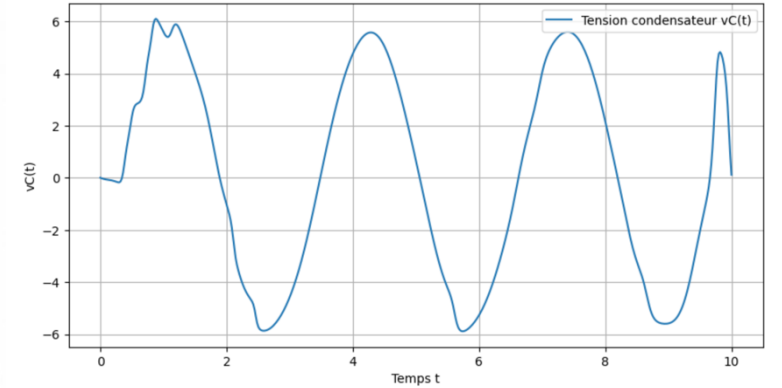
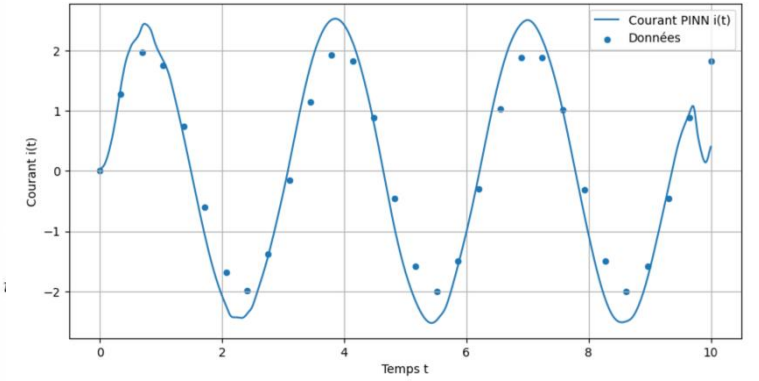
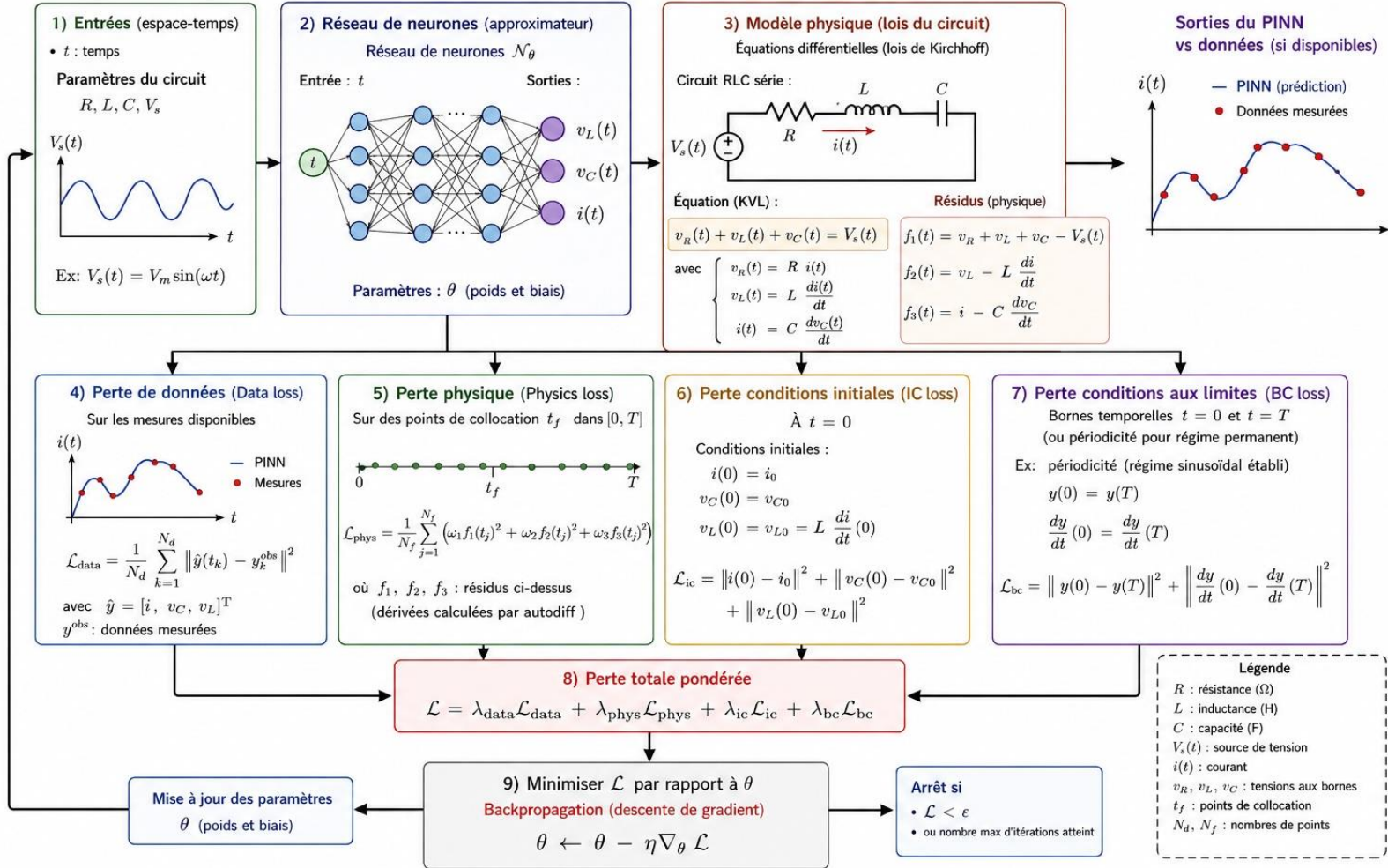
$\lambda_{\text{physics}} = 1$
 $\lambda_{\text{ic}} = 1$
 $\lambda_{\text{data}} = 1$

PINN pour l'équation de la chaleur 2D



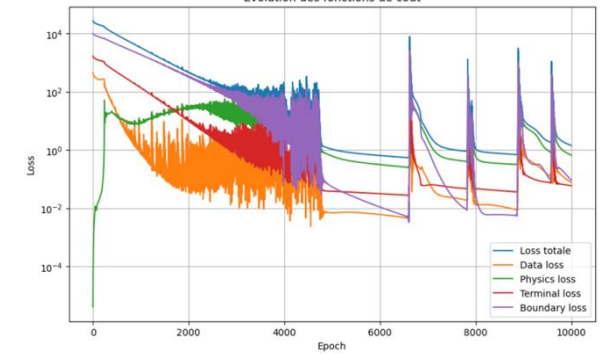
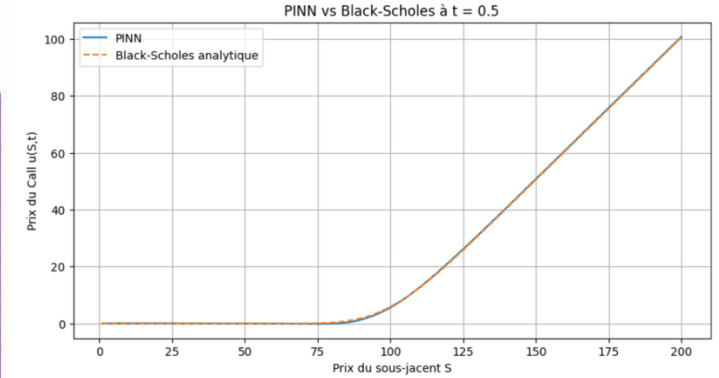
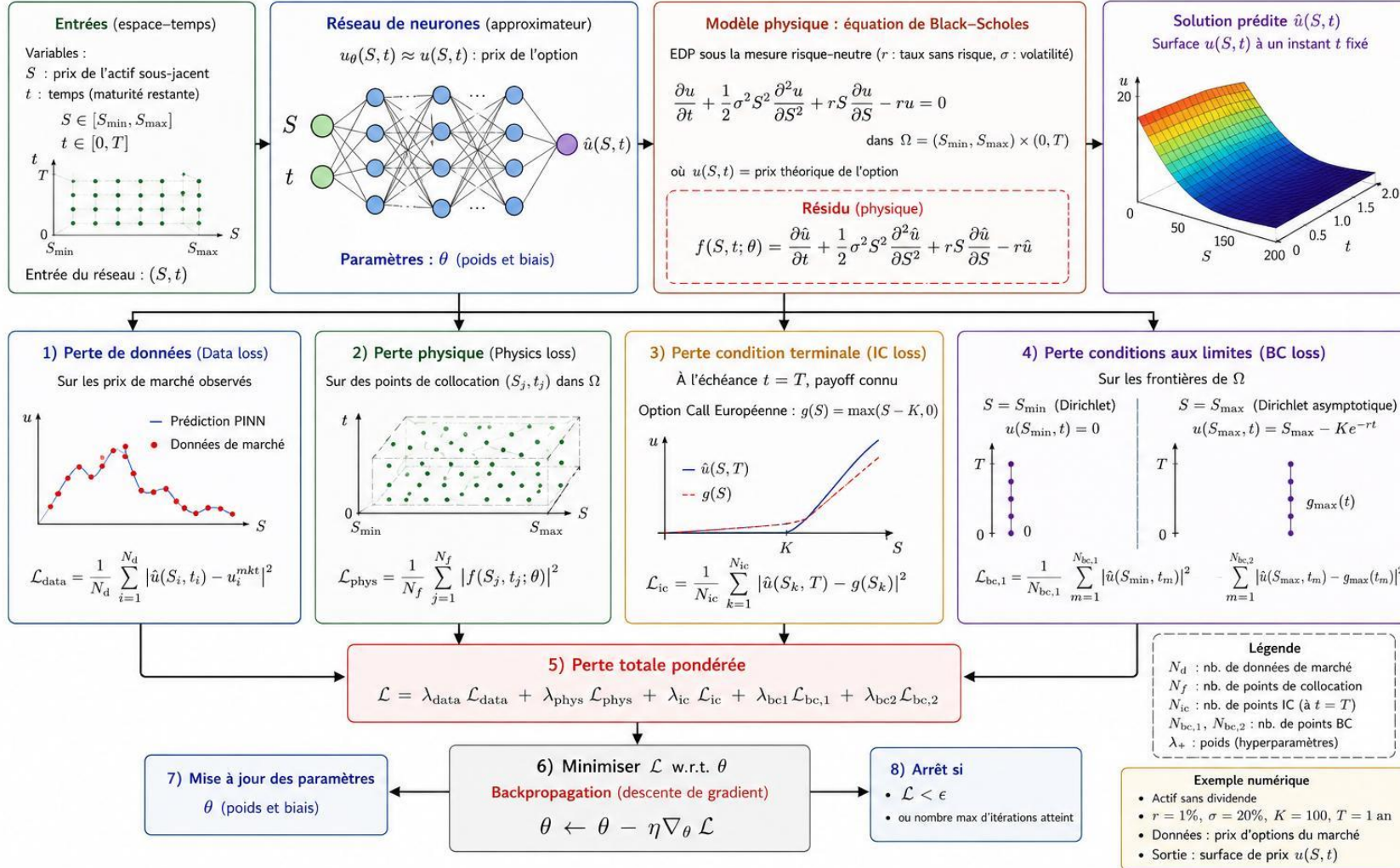
PINN pour un circuit électrique RLC série

Réseau de neurones informé par la physique

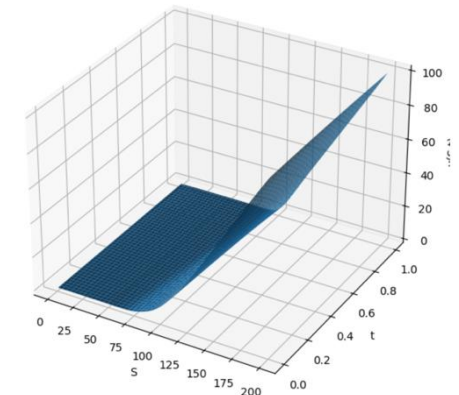


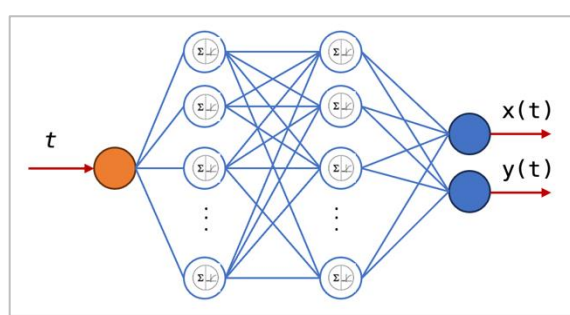
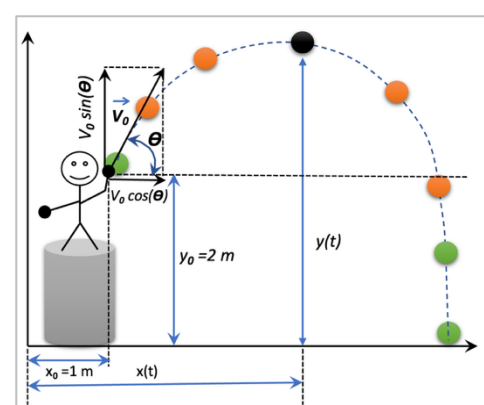
PINN pour la valorisation d'options Européennes (équation de Black-Scholes)

On apprend le prix de l'option $u(S, t)$ à partir des données du marché et des lois de la physique (absence d'arbitrage)



Surface prédite par le PINN





```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import math
```

Paramètres physiques

$g = 9.81$

Conditions initiales

$x_0 = 1.0$

$y_0 = 2.0$

$\theta = \pi/3$

$v_0 = 20$ # m/s

$v_{x0} = v_0 \cdot \cos(\theta)$

$v_{y0} = v_0 \cdot \sin(\theta)$

Temps de vol théorique

pour $y = 0$

$T_{\text{final}} = 2 \cdot v_{y0} / g$

```
# =====
# 2. Helper pour les dérivées
# =====
```

```
def derivative(y, t):
```

```
    """
```

```
    Calcule  $dy/dt$  avec autograd.
```

```
    """
```

```
    return torch.autograd.grad(
```

```
        y,
```

```
        t,
```

```
        grad_outputs=torch.ones_like(y),
```

```
        create_graph=True
```

```
    )[0]
```

```
# =====
# 3. Réseau PINN
# =====
```

```
pinn_model = nn.Sequential(
```

```
    nn.Linear(1, 64),
```

```
    nn.Tanh(),
```

```
    nn.Linear(64, 64),
```

```
    nn.Tanh(),
```

```
    nn.Linear(64, 2)
```

```
)
```

```
# =====
# 4. Points de collocation physique
# =====
```

```
N_f = 200
```

```
t_f = torch.linspace(0, T_final, N_f).view(-1, 1)
```

```
t_f.requires_grad_(True)
```

```
# =====
# 5. Condition initiale  $t = 0$ 
# =====
```

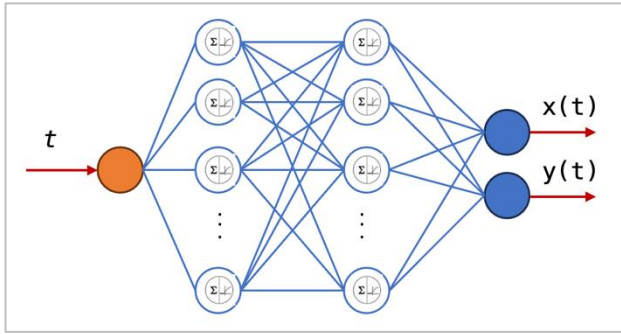
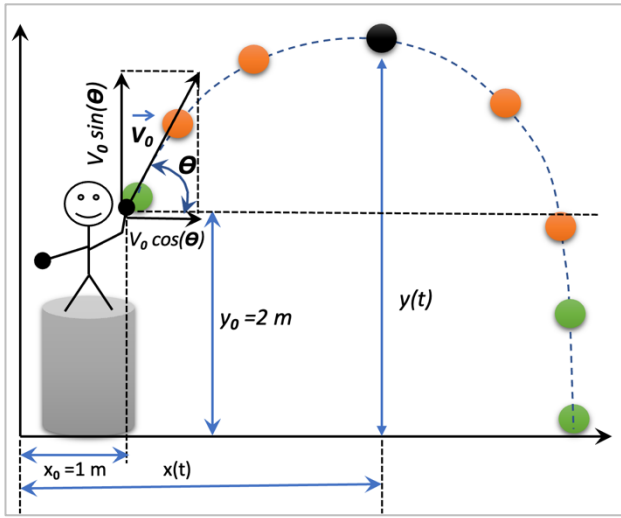
```
t0 = torch.tensor([[0.0]], dtype=torch.float32)
```

```
t0.requires_grad_(True)
```

```
# =====
# 7. Optimiseur
# =====
```

```
optimizer = optim.Adam(pinn_model.parameters(), lr=0.001)
```

```
# =====
# 8. Entraînement
# =====
```



$$\frac{d^2x}{dt^2} = 0$$

$$\frac{d^2y}{dt^2} = -g$$

$$x(0) = x_0, \quad y(0) = y_0$$

$$\frac{dx}{dt}(0) = v_0 \cos \theta$$

$$\frac{dy}{dt}(0) = v_0 \sin \theta$$

```

epochs = 8000
for epoch in range(epochs):
    pinn_model.train()
    # Loss physique
    pred_f = pinn_model(t_f)
    x_pred = pred_f[:, 0:1]
    y_pred = pred_f[:, 1:2]
    # Dérivées premières
    x_t = derivative(x_pred, t_f)
    y_t = derivative(y_pred, t_f)
    # Dérivées secondes
    x_tt = derivative(x_t, t_f)
    y_tt = derivative(y_t, t_f)
    # Equations physiques :
    # x''(t) = 0
    # y''(t) = -g => y''(t) + g = 0
    residual_x = x_tt
    residual_y = y_tt + g
    loss_physics = torch.mean(residual_x ** 2) + torch.mean(residual_y ** 2)
    # Loss conditions initiales
    pred0 = pinn_model(t0)
    x0_pred = pred0[:, 0:1]
    y0_pred = pred0[:, 1:2]
    x0_t = derivative(x0_pred, t0)
    y0_t = derivative(y0_pred, t0)
    loss_ic_position = (x0_pred - x0) ** 2 + (y0_pred - y0) ** 2
    loss_ic_velocity = (x0_t - vx0) ** 2 + (y0_t - vy0) ** 2
    loss_ic = torch.mean(loss_ic_position + loss_ic_velocity)
    # Loss totale
    loss = loss_physics + loss_ic
    # Backpropagation
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if (epoch + 1) % 1000 == 0:
        print( f"Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.6f} | » )

```

```
# =====
# 9. Évaluation
# =====
```

```
pinn_model.eval()
```

```
t_test = torch.linspace(0, T_final, 30).view(-1, 1)
```

```
with torch.no_grad():
    pred_test = pinn_model(t_test)
```

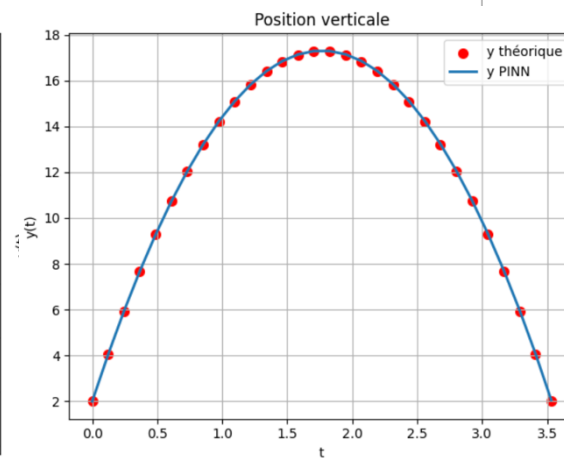
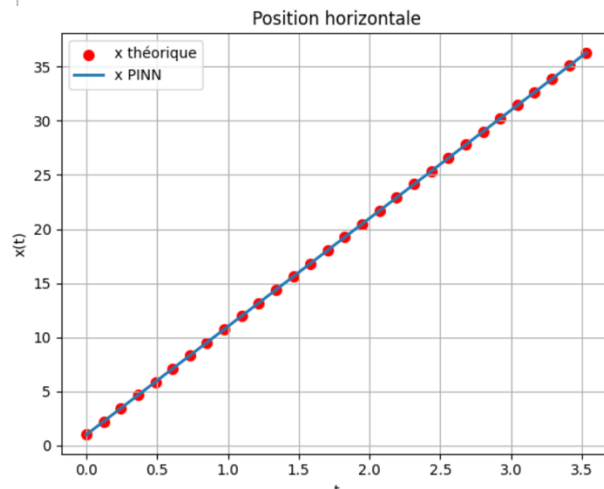
```
x_pinn = pred_test[:, 0:1]
```

```
y_pinn = pred_test[:, 1:2]
```

```
# Solution théorique exacte
```

```
x_exact = x0 + vx0 * t_test
```

```
y_exact = y0 + vy0 * t_test - 0.5 * g * t_test ** 2
```



```
# =====
# 11. Visualisation x(t) et y(t)
# =====
```

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
```

```
plt.scatter(t_test, x_exact, label="x théorique", color="red", linewidth=2)
```

```
plt.plot(t_test, x_pinn, label="x PINN", linewidth=2)
```

```
plt.xlabel("t")
```

```
plt.ylabel("x(t)")
```

```
plt.title("Position horizontale")
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.subplot(1, 2, 2)
```

```
plt.scatter(t_test, y_exact, label="y théorique", color="red", linewidth=2)
```

```
plt.plot(t_test, y_pinn, label="y PINN", linewidth=2)
```

```
plt.xlabel("t")
```

```
plt.ylabel("y(t)")
```

```
plt.title("Position verticale")
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.tight_layout()
```

```
plt.show()
```

```

# =====
# 12. Visualisation trajectoire y(x)
# =====

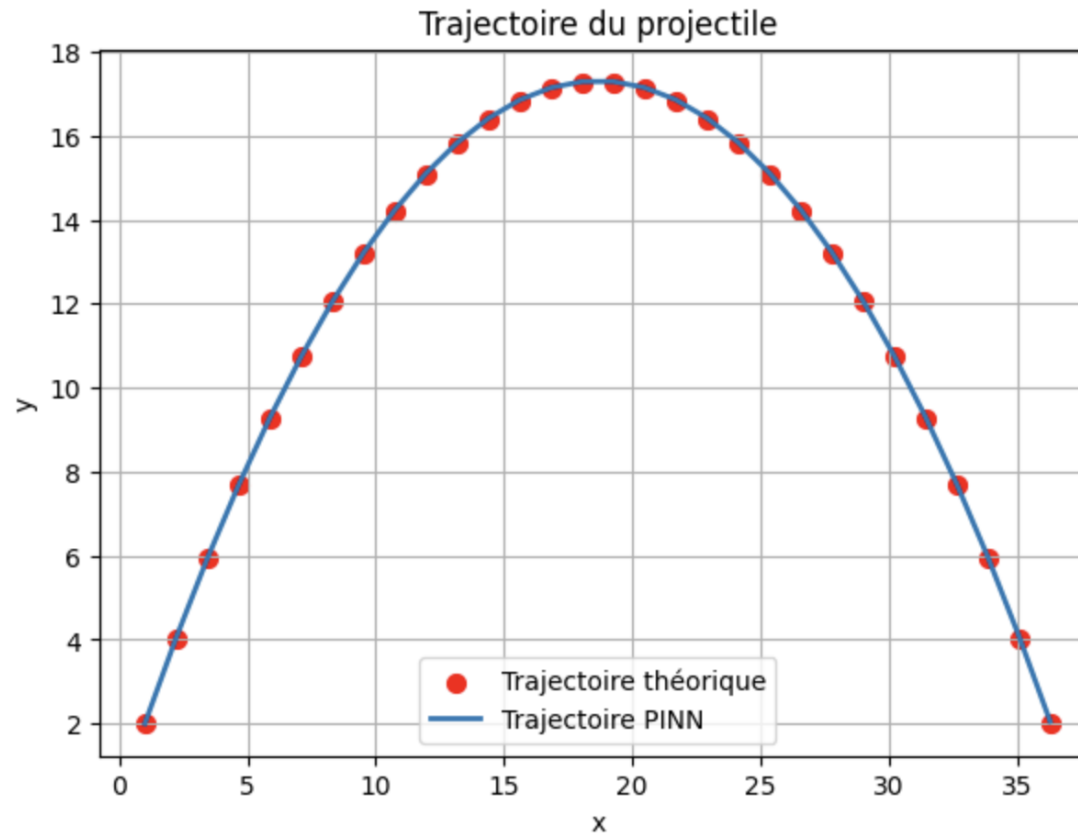
plt.figure(figsize=(7, 5))

plt.scatter(
    x_exact,
    y_exact,
    label="Trajectoire théorique",
    linewidth=2,
    color="red"
)

plt.plot(
    x_pinn,
    y_pinn,
    label="Trajectoire PINN",
    linewidth=2
)

plt.xlabel("x")
plt.ylabel("y")
plt.title("Trajectoire du projectile")
plt.legend()
plt.grid()
plt.show()

```



Implémentation des PINNs

```
[project]
name = "pinns"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
    "ipykernel>=7.1.0",
    "keras>=3.13.2",
    "matplotlib>=3.10.8",
    "pandas>=3.0.0",
    "scikit-learn>=1.8.0",
    "tensorflow>=2.20.0",
    "torch>=2.10.0",
    "torchvision>=0.25.0",
    "torchviz>=0.0.3",
]
```

```
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from math import sin, pi
```

Implémentation des PINNs : Generate Synthetic Data

Physics parameters

$g = 9.8$ # acceleration due to gravity

$h_0 = 1.0$ # initial height

$v_0 = 13.5$ # initial velocity

$\alpha = \pi/4$

True (analytical) solution $h(t) = h_0 + v_0 t - 0.5 g t^2$

def true_solution(t):

 return $h_0 + v_0 \sin(\alpha) t - 0.5 g t^2$

Generate some time points

$t_{\min}, t_{\max} = 0.0, 2.0$

$N_{\text{data}} = 10$

$t_{\text{data}} = \text{np.linspace}(t_{\min}, t_{\max}, N_{\text{data}})$

Generate synthetic "experimental" heights with noise

$\text{np.random.seed}(0)$

$\text{noise_level} = 0.5$

$h_{\text{data_exact}} = \text{true_solution}(t_{\text{data}})$

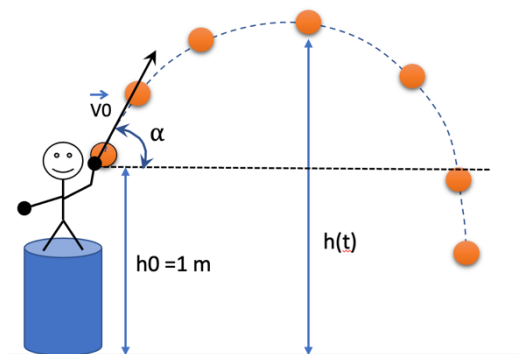
$h_{\text{data_noisy}} = h_{\text{data_exact}} + \text{noise_level} * \text{np.random.randn}(N_{\text{data}})$

Convert to PyTorch tensors

$t_{\text{data_tensor}} = \text{torch.tensor}(t_{\text{data}}, \text{dtype}=\text{torch.float32}).\text{view}(-1, 1)$

$h_{\text{data_tensor}} = \text{torch.tensor}(h_{\text{data_noisy}}, \text{dtype}=\text{torch.float32}).\text{view}(-1, 1)$

$\text{pd.DataFrame}(\{ "t": t_{\text{data}}, "h_Exp": h_{\text{data_noisy}}, "h_Physics": h_{\text{data_exact}} \})$



$$\frac{dh}{dt} = v_0 \sin \alpha - g t$$

$$h = -\frac{1}{2} g t^2 + v_0 \sin \alpha t + h_0$$

v_0 : Initial velocity at angle α

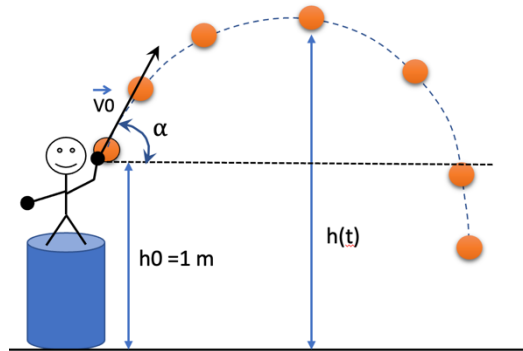
g : Acceleration due to gravity

	t	h_Exp	h_Physics
0	0.000000	1.882026	1.000000
1	0.222222	3.079424	2.879345
2	0.444444	4.764108	4.274739
3	0.666667	6.306630	5.186183
4	0.888889	6.547455	5.613676
5	1.111111	5.068580	5.557219
6	1.333333	5.491855	5.016811
7	1.555556	3.916774	3.992452
8	1.777778	2.432534	2.484143
9	2.000000	0.697182	0.491883

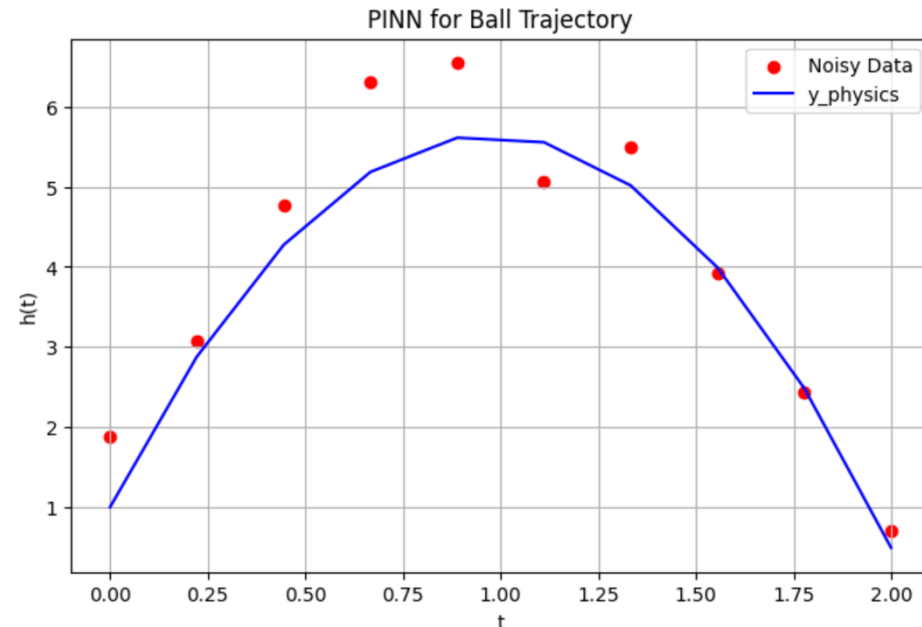
Implémentation des PINNs : Plot Results

Plot results

```
plt.figure(figsize=(8, 5))
plt.scatter(t_data, h_data_noisy, color='red', label='Noisy Data')
plt.plot(t_data, h_data_exact, color = 'blue', label = "y_physics")
plt.xlabel('t')
plt.ylabel('h(t)')
plt.legend()
plt.title('PINN for Ball Trajectory')
plt.grid(True)
plt.show()
```



	t	h_Exp	h_Physics
0	0.000000	1.882026	1.000000
1	0.222222	3.079424	2.879345
2	0.444444	4.764108	4.274739
3	0.666667	6.306630	5.186183
4	0.888889	6.547455	5.613676
5	1.111111	5.068580	5.557219
6	1.333333	5.491855	5.016811
7	1.555556	3.916774	3.992452
8	1.777778	2.432534	2.484143
9	2.000000	0.697182	0.491883

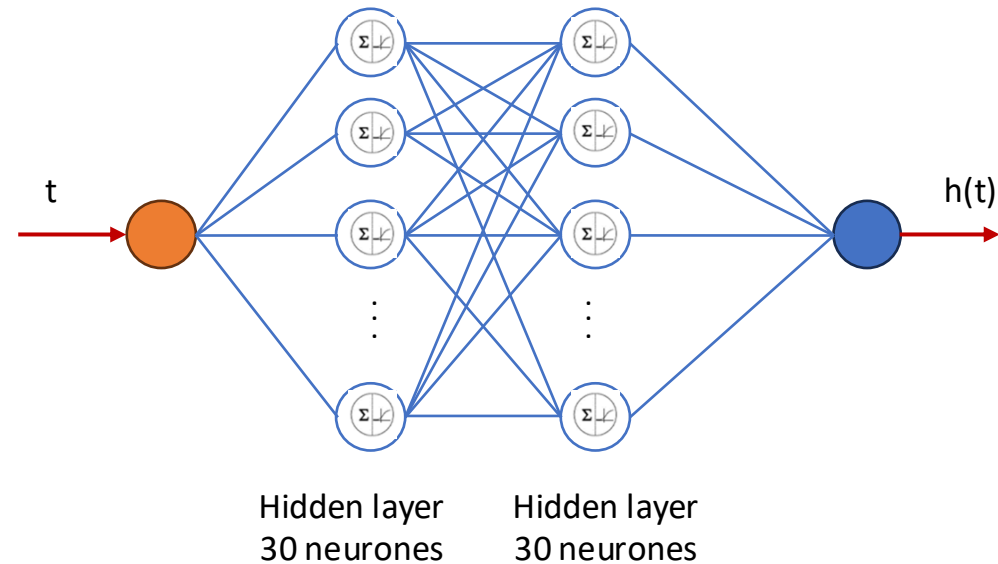


PINNs Implementation : Pytorch Neural Network Model

```
class PINN(nn.Module):  
    def __init__(self, n_hidden=20):  
        super(PINN, self).__init__()  
        # A simple MLP with 2 hidden layers  
        self.net = nn.Sequential(  
            nn.Linear(1, n_hidden),  
            nn.Tanh(),  
            nn.Linear(n_hidden, n_hidden),  
            nn.Tanh(),  
            nn.Linear(n_hidden, 1)  
        )  
  
    def forward(self, t):  
        return self.net(t)
```

```
model_pinns = PINN(n_hidden=30)
```

```
PINN(  
    (net): Sequential(  
      (0): Linear(in_features=1, out_features=30, bias=True)  
      (1): Tanh()  
      (2): Linear(in_features=30, out_features=30, bias=True)  
      (3): Tanh()  
      (4): Linear(in_features=30, out_features=1, bias=True)  
    )  
)
```

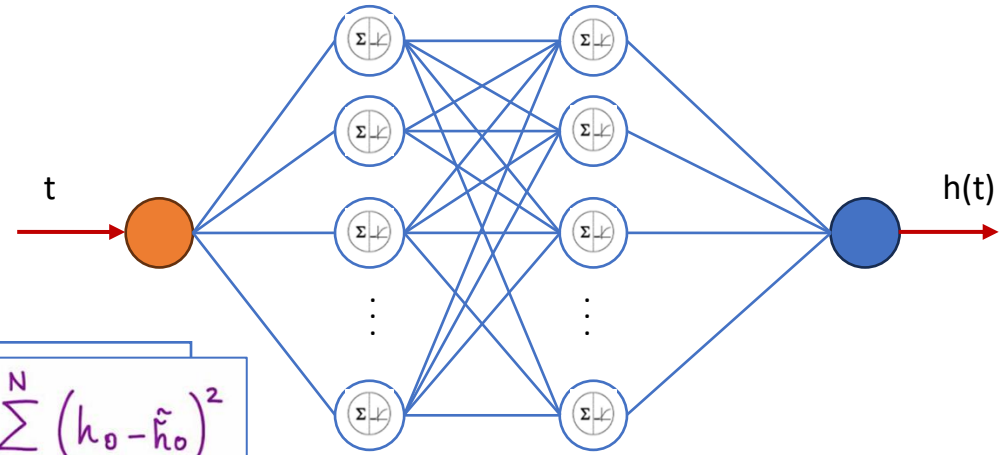


$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

PINNs Implementation : Loss Functions

```
def derivative(y, x):
    return torch.autograd.grad(
        y, x,
        grad_outputs=torch.ones_like(y),
        create_graph=True
    )[0]
```

```
def data_loss(model, t_data, h_data):
    h_pred = model(t_data)
    return torch.mean((h_pred - h_data)**2)
```



$$\frac{1}{N} \sum_{i=1}^N (h_0 - \tilde{h}_0)^2$$

```
def initial_condition_loss(model):
    t0 = torch.zeros(1, 1, dtype=torch.float32, requires_grad=False)
    h0_pred = model(t0)
    return (h0_pred - h0).pow(2).mean()
```

$$\frac{1}{N} \sum_{i=1}^N (h_0 - \tilde{h}_0)^2$$

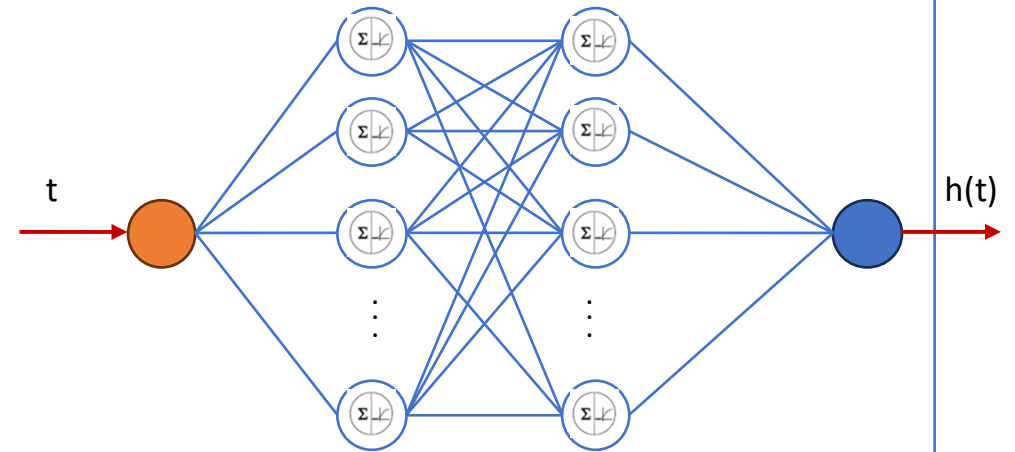
```
def physics_loss(model, t):
    t.requires_grad_(True)
    h_pred = model(t)
    dh_dt_pred = derivative(h_pred, t)
    dh_dt_true = v0 * sin(alpha) - g * t
    loss_ode = torch.mean((dh_dt_pred - dh_dt_true)**2)
    return loss_ode
```

$$\frac{1}{N} \sum_{i=1}^N \left(v_0 \sin \alpha - g t_i - d \frac{\hat{h}_i}{dt} \right)^2$$

PINNs Implementation : Model Training

```
lambda_data = 2.0  
lambda_ode = 8.0  
lambda_ic = 8.0  
num_epochs = 8000  
print_every = 1000
```

```
def train (model, with_pinn=True):  
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)  
    model.train()  
    for epoch in range(num_epochs):  
        optimizer.zero_grad()  
        l_data = data_loss(model, t_data_tensor, h_data_tensor)  
        if with_pinn == True:  
            l_ode = physics_loss(model, t_data_tensor)  
            l_ic = initial_condition_loss(model)  
            loss = lambda_data * l_data + lambda_ode * l_ode + lambda_ic * l_ic  
        else :  
            loss = l_data  
        # Backprop  
        loss.backward()  
        optimizer.step()  
        # Print progress  
        if (epoch+1) % print_every == 0:  
            print(f"Epoch {epoch+1}/{num_epochs}, "  
                  f"Total Loss = {loss.item():.6f}, ")
```



PINNs Implementation : Model Training

```
# Instantiate models
```

```
model_no_pinns = PINN(n_hidden=30)
```

```
model_pinns = PINN(n_hidden=30)
```

```
print("="*60)
```

```
print("Train Model without pinns")
```

```
print("="*60)
```

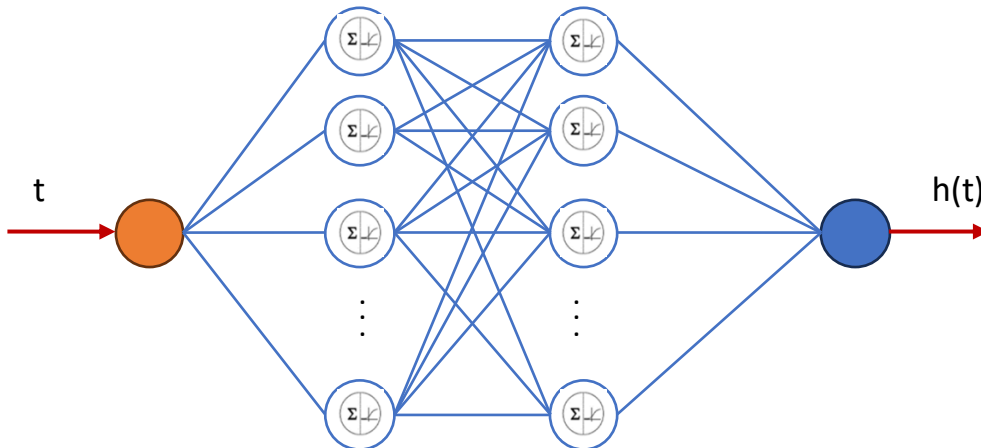
```
train(model_no_pinns, with_pinn=False)
```

```
print("="*60)
```

```
print("Train Model With Pinns")
```

```
print("="*60)
```

```
train(model_pinns, with_pinn=True)
```



```
=====
```

```
Train Model without pinns
```

```
=====
```

```
Epoch 1000/8000, Total Loss = 0.125508,
```

```
Epoch 2000/8000, Total Loss = 0.079400,
```

```
Epoch 3000/8000, Total Loss = 0.061639,
```

```
Epoch 4000/8000, Total Loss = 0.024623,
```

```
Epoch 5000/8000, Total Loss = 0.000163,
```

```
Epoch 6000/8000, Total Loss = 0.000000,
```

```
Epoch 7000/8000, Total Loss = 0.000000,
```

```
Epoch 8000/8000, Total Loss = 0.000000,
```

```
=====
```

```
Train Model With Pinns
```

```
=====
```

```
Epoch 1000/8000, Total Loss = 0.630344,
```

```
Epoch 2000/8000, Total Loss = 0.553990,
```

```
Epoch 3000/8000, Total Loss = 0.552962,
```

```
Epoch 4000/8000, Total Loss = 0.551791,
```

```
Epoch 5000/8000, Total Loss = 0.549840,
```

```
Epoch 6000/8000, Total Loss = 0.548039,
```

```
Epoch 7000/8000, Total Loss = 0.542464,
```

```
Epoch 8000/8000, Total Loss = 0.300015,
```

PINNs Implementation : Model Evaluation

```
model_pinns.eval()
```

```
model_no_pinns.eval()
```

```
t_plot = np.linspace(t_min, t_max, 100).reshape(-1, 1).astype(np.float32)
t_plot_tensor = torch.tensor(t_plot, requires_grad=True)
# Data Driven Model Prediction
h_pred_no_pinns_plot = model_no_pinns(t_plot_tensor).detach().numpy()
# Pinns Model Prediction
h_pred_pinns_plot = model_pinns(t_plot_tensor).detach().numpy()
# True solution (for comparison)
h_true_plot = true_solution(t_plot)
# Plot results
plt.figure(figsize=(8, 5))
plt.scatter(t_data, h_data_noisy, color='red', label='Noisy Data')
plt.plot(t_plot, h_true_plot, 'k--', label='Exact Solution')
plt.plot(t_plot, h_pred_pinns_plot, 'b', label='PINN Prediction')
plt.plot(t_plot, h_pred_no_pinns_plot, 'r', label='No PINNs Prediction')
plt.xlabel('t')
plt.ylabel('h(t)')
plt.legend()
plt.title('PINN for Ball Trajectory')
plt.grid(True)
plt.show()
```

